Lecture 31

Greedy: Kruskal's Algorithm (contd.), Prim's Algorithm

Source: Introduction to Algorithms, CLRS

Kruskal's Algorithm: Correctness (Minimum)

Kruskal's Algorithm: Correctness (Minimum)

Now, we will prove that at any stage of the algorithm, A is part of some MST.

Kruskal's Algorithm: Correctness (Minimum)

Now, we will prove that at any stage of the algorithm, A is part of some MST.

Initially, $A = \emptyset$ is trivially a part of every MST.

Kruskal's Algorithm: Correctness (Minimum)

Now, we will prove that at any stage of the algorithm, A is part of some MST.

Initially, $A = \emptyset$ is trivially a part of every MST.

Now suppose, edge $\{u, v\}$ is about to be added to A

Kruskal's Algorithm: Correctness (Minimum)

Now, we will prove that at any stage of the algorithm, A is part of some MST.

Initially, $A = \emptyset$ is trivially a part of every MST.

Now suppose, edge $\{u, v\}$ is about to be added to A connecting two subtrees, say T_i and T_j.

Kruskal's Algorithm: Correctness (Minimum)

Now, we will prove that at any stage of the algorithm, A is part of some MST.

Initially, $A = \emptyset$ is trivially a part of every MST.

Now suppose, edge $\{u, v\}$ is about to be added to A connecting two subtrees, say T_i and T_j.

We will prove that $A \cup \{u, v\}$ will also be a part of some MST.

Kruskal's Algorithm: Correctness (Minimum)

Now, we will prove that at any stage of the algorithm, A is part of some MST.

Initially, $A = \emptyset$ is trivially a part of every MST.

Now suppose, edge $\{u, v\}$ is about to be added to A connecting two subtrees, say T_i and T_j.

We will prove that $A \cup \{u, v\}$ will also be a part of some MST.

Consider the cut formed by vertices of T_i and the remaining vertices:

Kruskal's Algorithm: Correctness (Minimum)

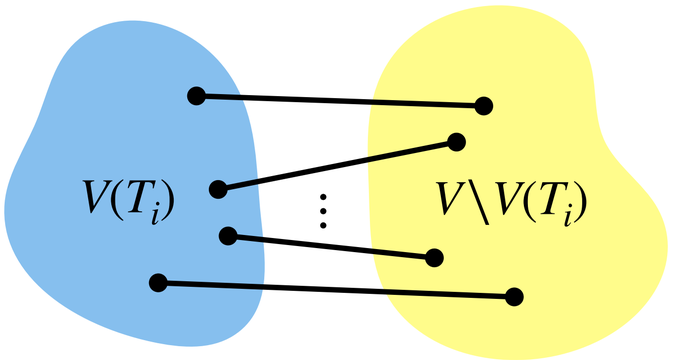
Now, we will prove that at any stage of the algorithm, A is part of some MST.

Initially, $A = \emptyset$ is trivially a part of every MST.

Now suppose, edge $\{u, v\}$ is about to be added to A connecting two subtrees, say T_i and T_j.

We will prove that $A \cup \{u, v\}$ will also be a part of some MST.

Consider the cut formed by vertices of T_i and the remaining vertices:



Kruskal's Algorithm: Correctness (Minimum)

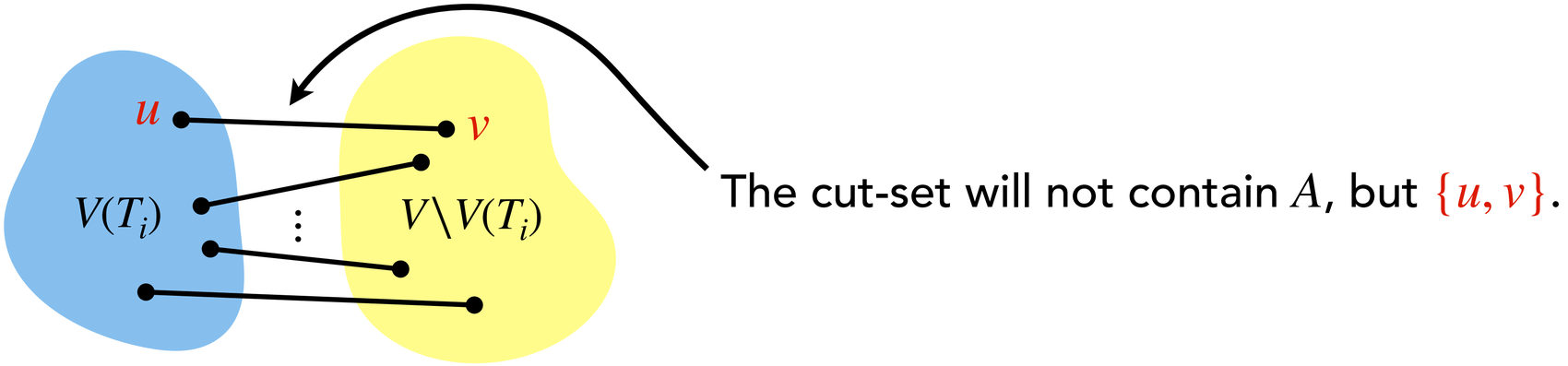
Now, we will prove that at any stage of the algorithm, A is part of some MST.

Initially, $A = \emptyset$ is trivially a part of every MST.

Now suppose, edge $\{u, v\}$ is about to be added to A connecting two subtrees, say T_i and T_j.

We will prove that $A \cup \{u, v\}$ will also be a part of some MST.

Consider the cut formed by vertices of T_i and the remaining vertices:



Kruskal's Algorithm: Correctness (Minimum)

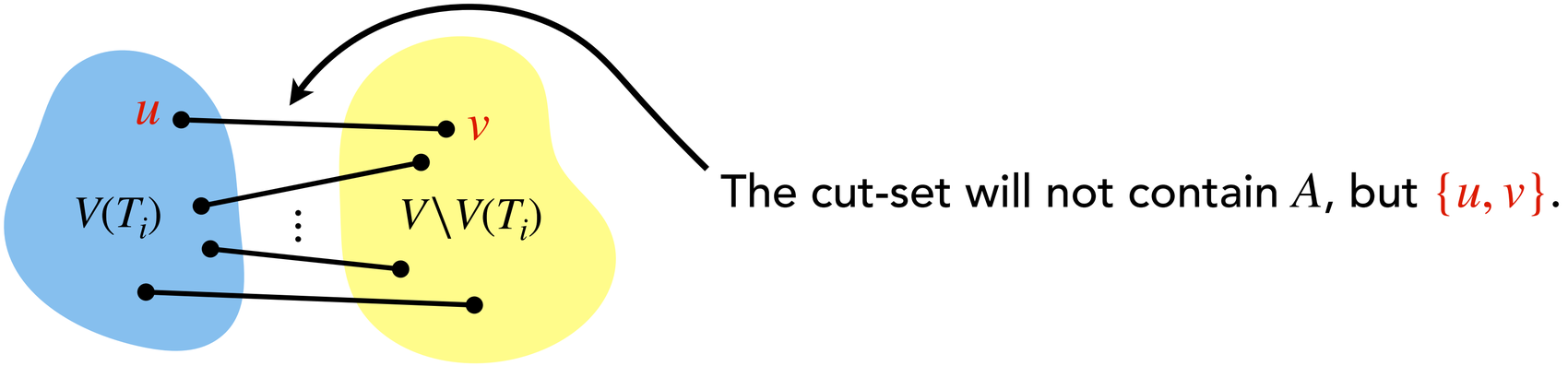
Now, we will prove that at any stage of the algorithm, A is part of some MST.

Initially, $A = \emptyset$ is trivially a part of every MST.

Now suppose, edge $\{u, v\}$ is about to be added to A connecting two subtrees, say T_i and T_j.

We will prove that $A \cup \{u, v\}$ will also be a part of some MST.

Consider the cut formed by vertices of T_i and the remaining vertices:



If $\{u, v\}$ is a least weight edge in the above cut-set, then we are done.

Kruskal's Algorithm: Correctness (Minimum)

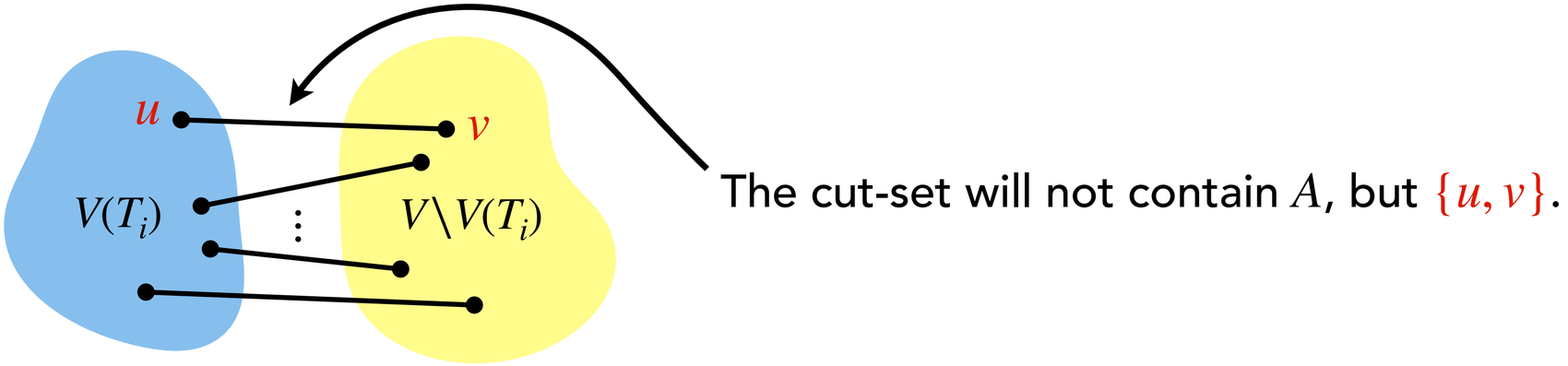
Now, we will prove that at any stage of the algorithm, A is part of some MST.

Initially, $A = \emptyset$ is trivially a part of every MST.

Now suppose, edge $\{u, v\}$ is about to be added to A connecting two subtrees, say T_i and T_j.

We will prove that $A \cup \{u, v\}$ will also be a part of some MST.

Consider the cut formed by vertices of T_i and the remaining vertices:



Kruskal's Algorithm: Correctness (Minimum)

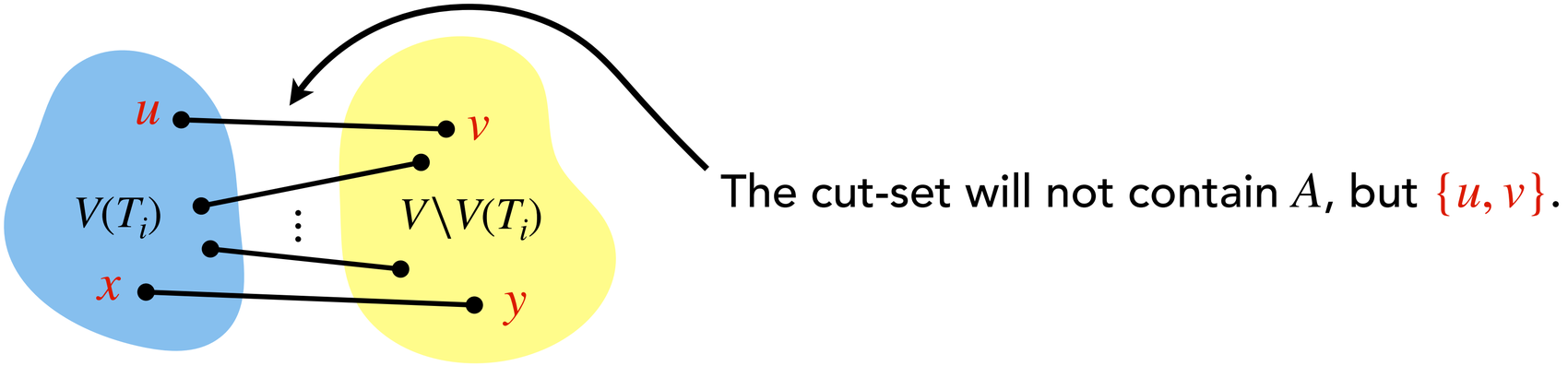
Now, we will prove that at any stage of the algorithm, A is part of some MST.

Initially, $A = \emptyset$ is trivially a part of every MST.

Now suppose, edge $\{u, v\}$ is about to be added to A connecting two subtrees, say T_i and T_j.

We will prove that $A \cup \{u, v\}$ will also be a part of some MST.

Consider the cut formed by vertices of T_i and the remaining vertices:



If $\{x, y\}$ not $\{u, v\}$ is a least weight edge of the cut-set,

Kruskal's Algorithm: Correctness (Minimum)

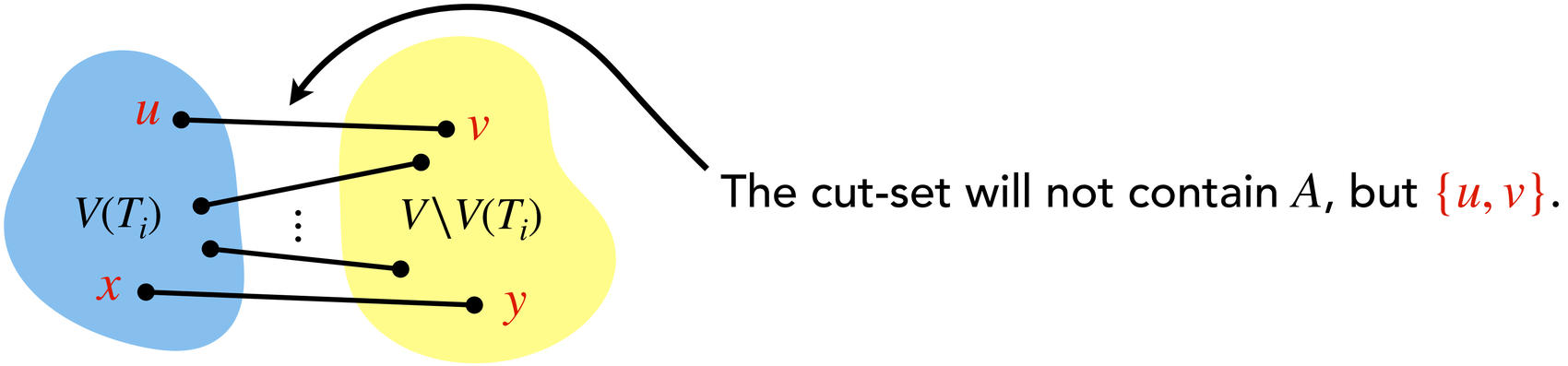
Now, we will prove that at any stage of the algorithm, A is part of some MST.

Initially, $A = \emptyset$ is trivially a part of every MST.

Now suppose, edge $\{u, v\}$ is about to be added to A connecting two subtrees, say T_i and T_j.

We will prove that $A \cup \{u, v\}$ will also be a part of some MST.

Consider the cut formed by vertices of T_i and the remaining vertices:



If $\{x, y\}$ not $\{u, v\}$ is a least weight edge of the cut-set, then algorithm would have considered

Kruskal's Algorithm: Correctness (Minimum)

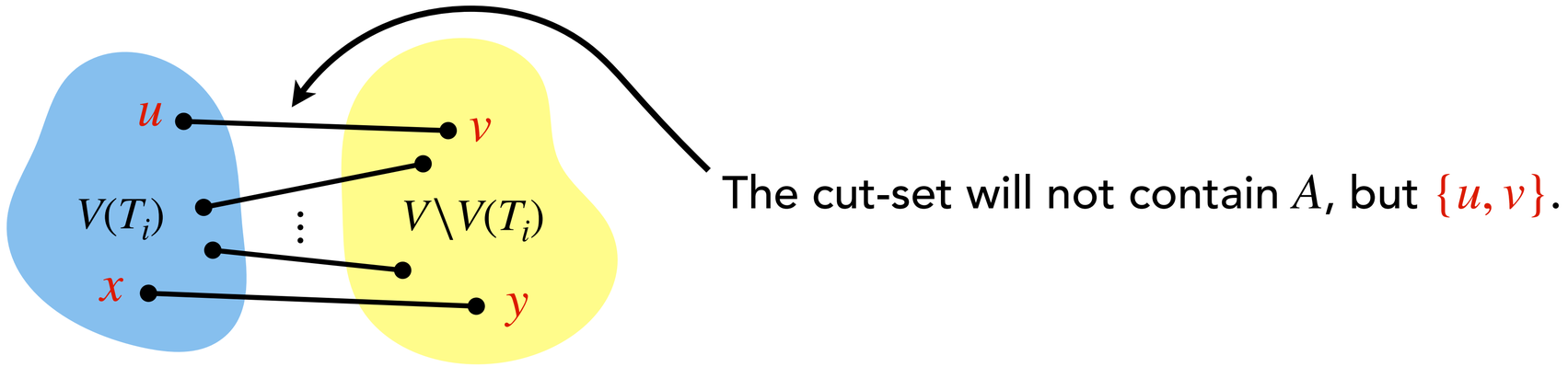
Now, we will prove that at any stage of the algorithm, A is part of some MST.

Initially, $A = \emptyset$ is trivially a part of every MST.

Now suppose, edge $\{u, v\}$ is about to be added to A connecting two subtrees, say T_i and T_j.

We will prove that $A \cup \{u, v\}$ will also be a part of some MST.

Consider the cut formed by vertices of T_i and the remaining vertices:



If $\{x, y\}$ not $\{u, v\}$ is a least weight edge of the cut-set, then algorithm would have considered $\{x, y\}$ before $\{u, v\}$and have added $\{x, y\}$ to A

Kruskal's Algorithm: Correctness (Minimum)

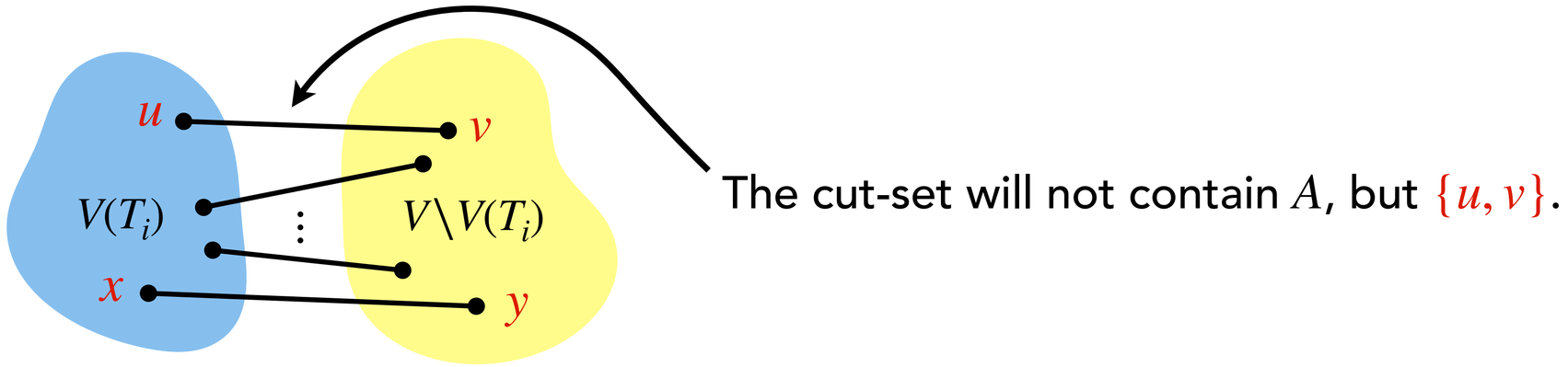
Now, we will prove that at any stage of the algorithm, A is part of some MST.

Initially, $A = \emptyset$ is trivially a part of every MST.

Now suppose, edge $\{u, v\}$ is about to be added to A connecting two subtrees, say T_i and T_j.

We will prove that $A \cup \{u, v\}$ will also be a part of some MST.

Consider the cut formed by vertices of T_i and the remaining vertices:



If $\{x, y\}$ not $\{u, v\}$ is a least weight edge of the cut-set, then algorithm would have considered $\{x, y\}$ before $\{u, v\}$and have added $\{x, y\}$ to A putting x and y in same subtree.

Kruskal's Algorithm: Correctness (Minimum)

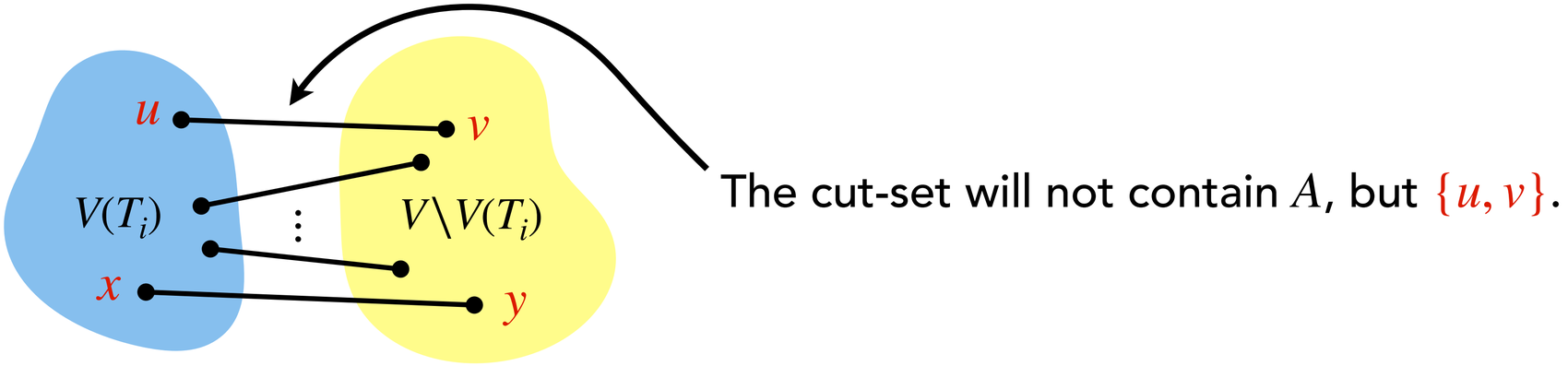
Now, we will prove that at any stage of the algorithm, A is part of some MST.

Initially, $A = \emptyset$ is trivially a part of every MST.

Now suppose, edge $\{u, v\}$ is about to be added to A connecting two subtrees, say T_i and T_j.

We will prove that $A \cup \{u, v\}$ will also be a part of some MST.

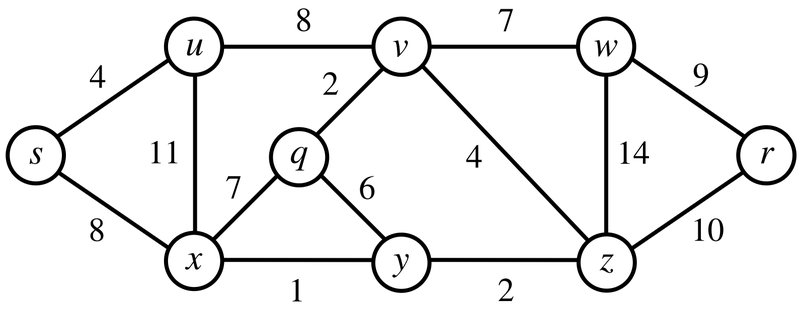
Consider the cut formed by vertices of T_i and the remaining vertices:



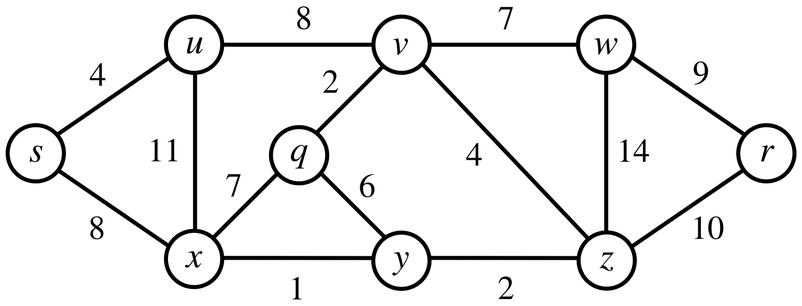
If $\{x, y\}$ not $\{u, v\}$ is a least weight edge of the cut-set, then algorithm would have considered $\{x, y\}$ before $\{u, v\}$and have added $\{x, y\}$ to A putting x and y in same subtree. Contradiction!

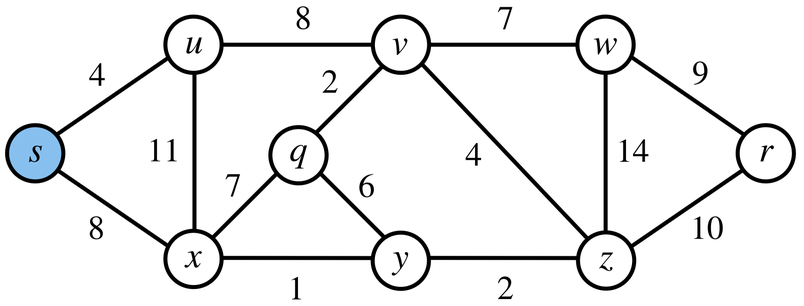
Prim’s Algorithm: Demonstration

Prim's Algorithm: Demonstration

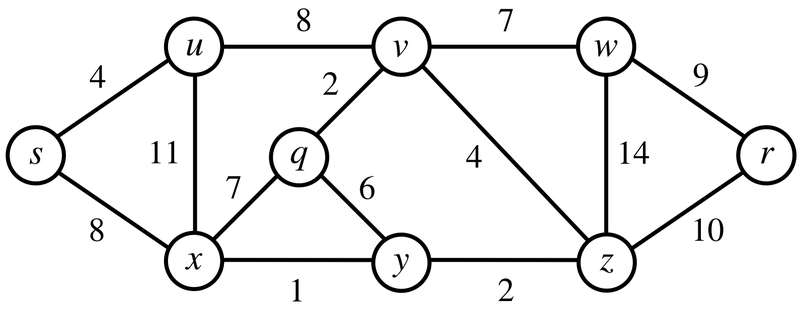


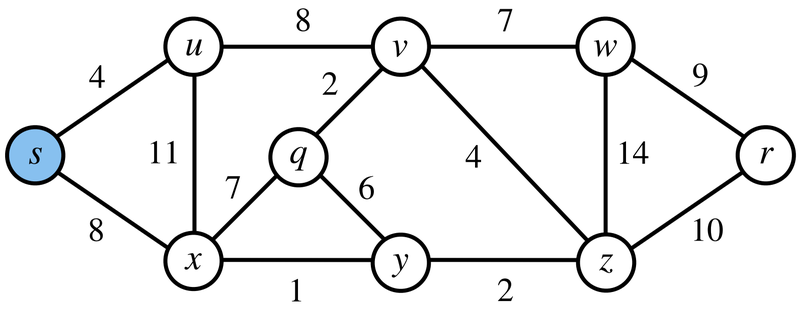
Prim's Algorithm: Demonstration





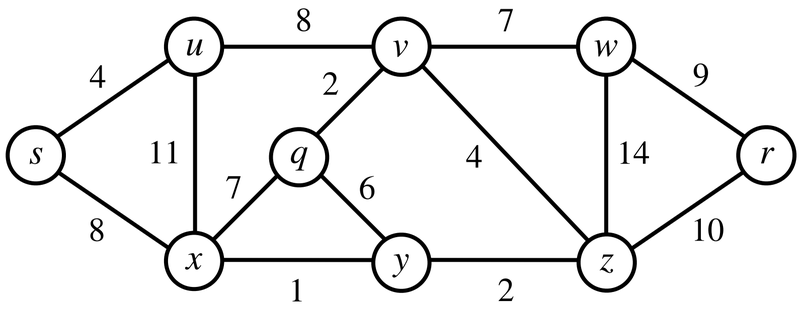
Prim's Algorithm: Demonstration

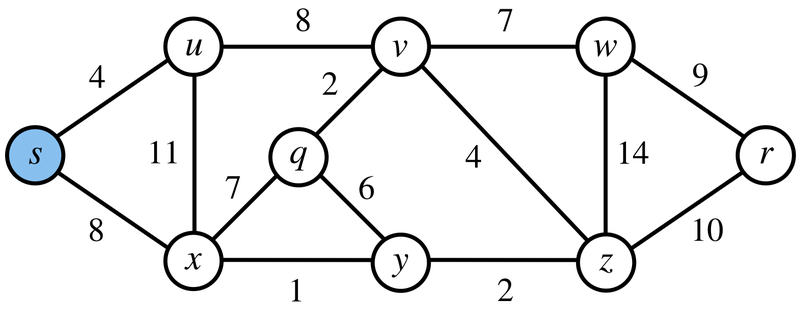




Build a "rooted" spanning tree step by step by repeatedly adding a least weight edge

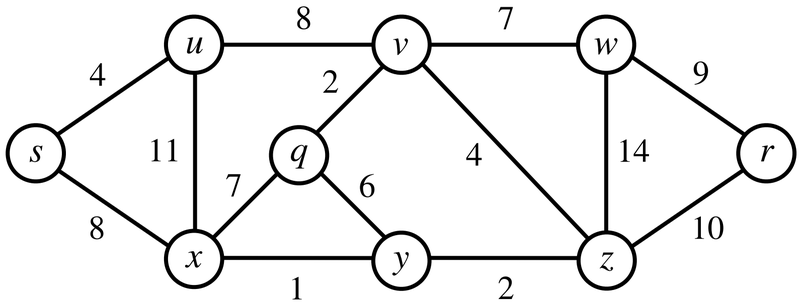
Prim's Algorithm: Demonstration

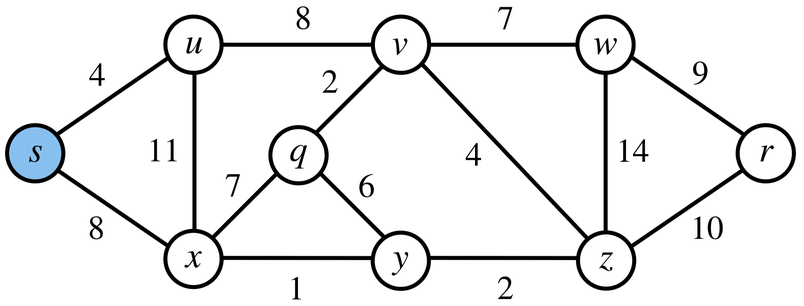




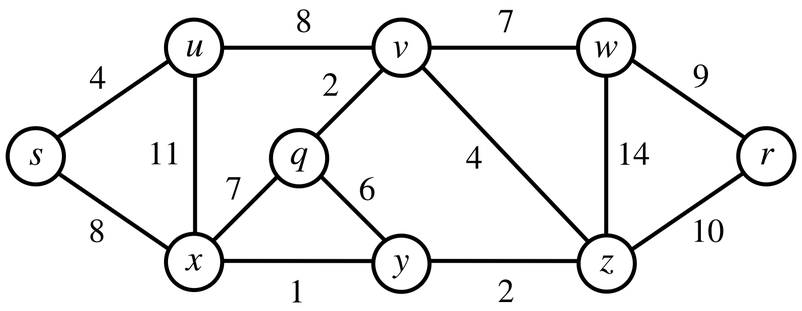
Build a "rooted" spanning tree step by step by repeatedly adding a least weight edge from the cut formed by the vertices of the tree and the remaining vertices.

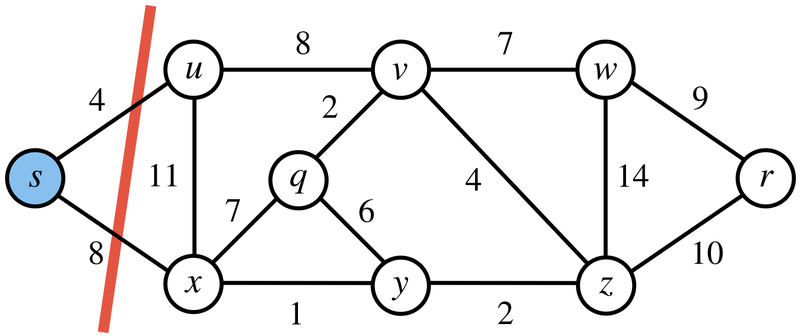
Prim's Algorithm: Demonstration



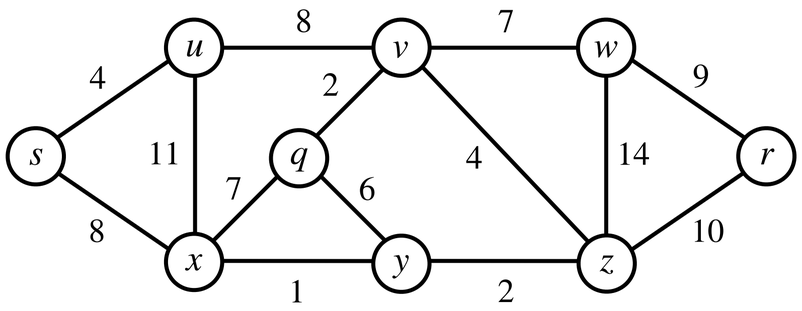


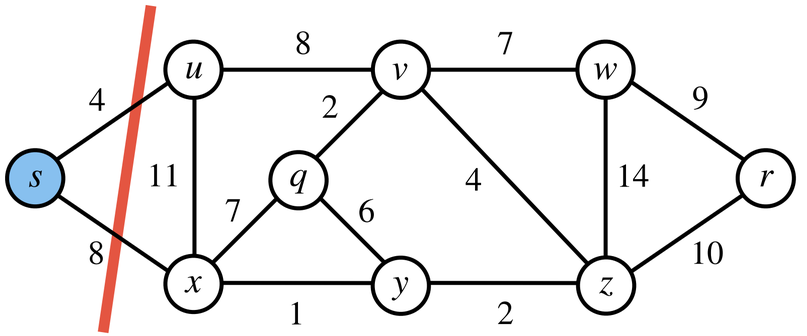
Prim's Algorithm: Demonstration

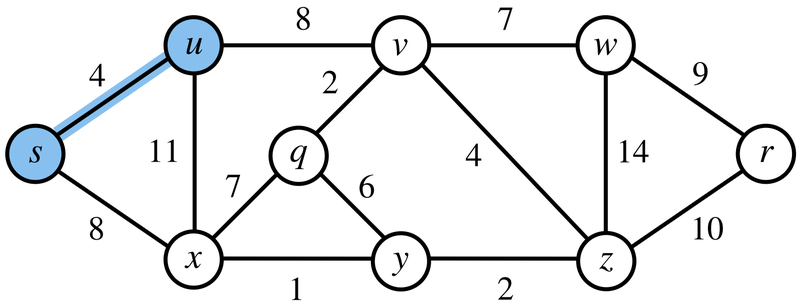




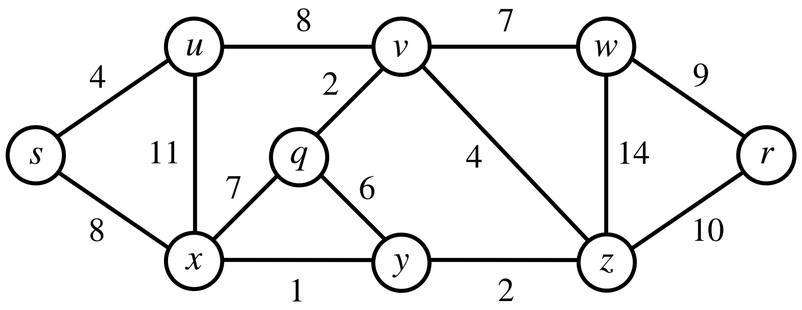
Prim's Algorithm: Demonstration

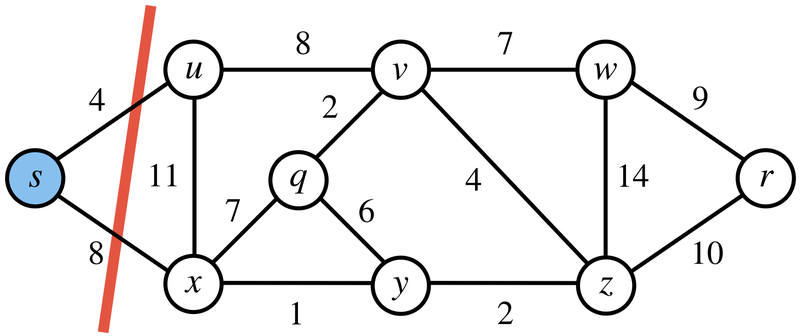


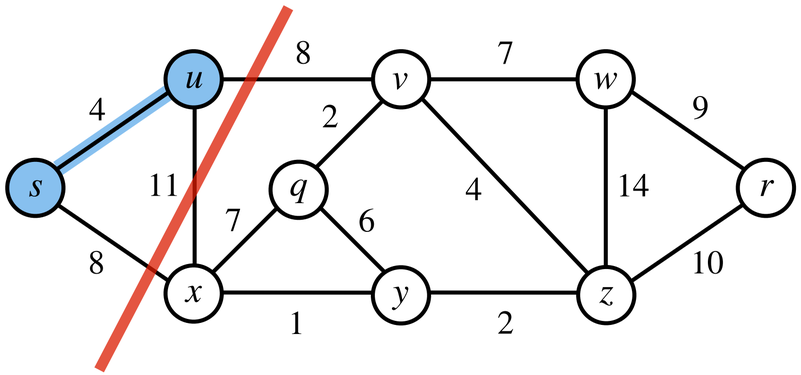




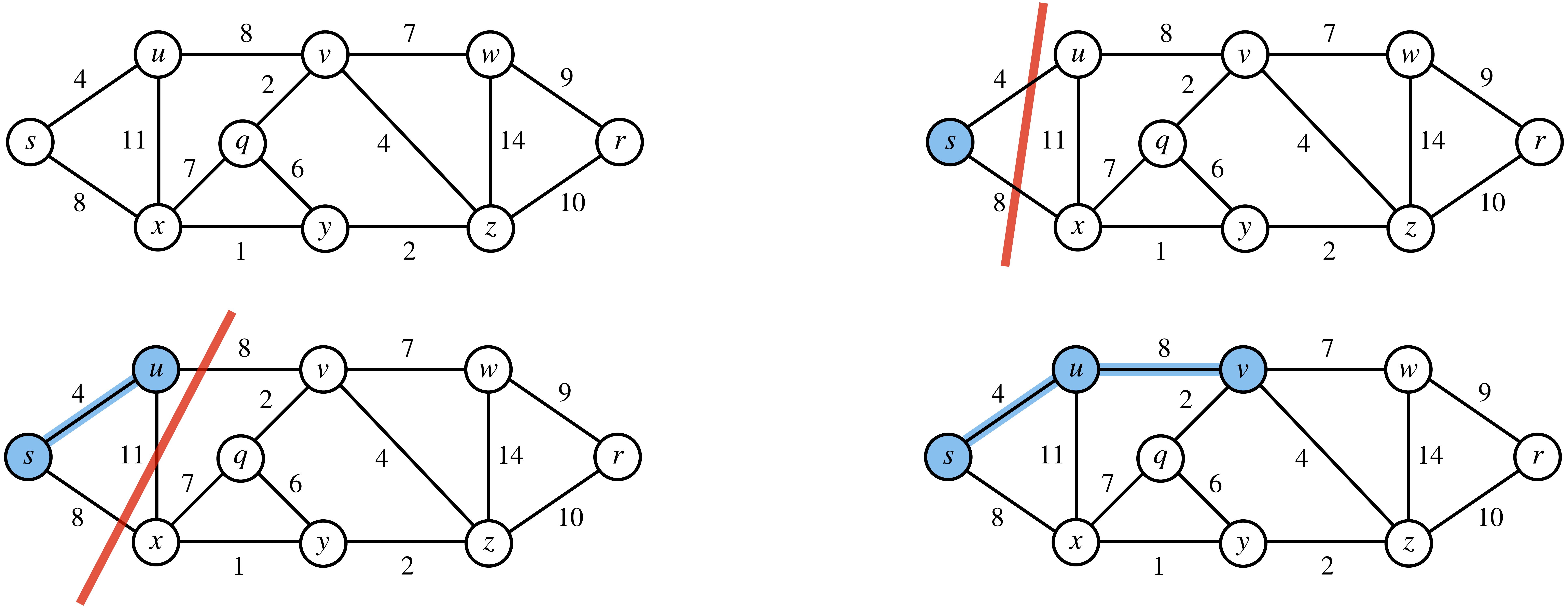
Prim's Algorithm: Demonstration



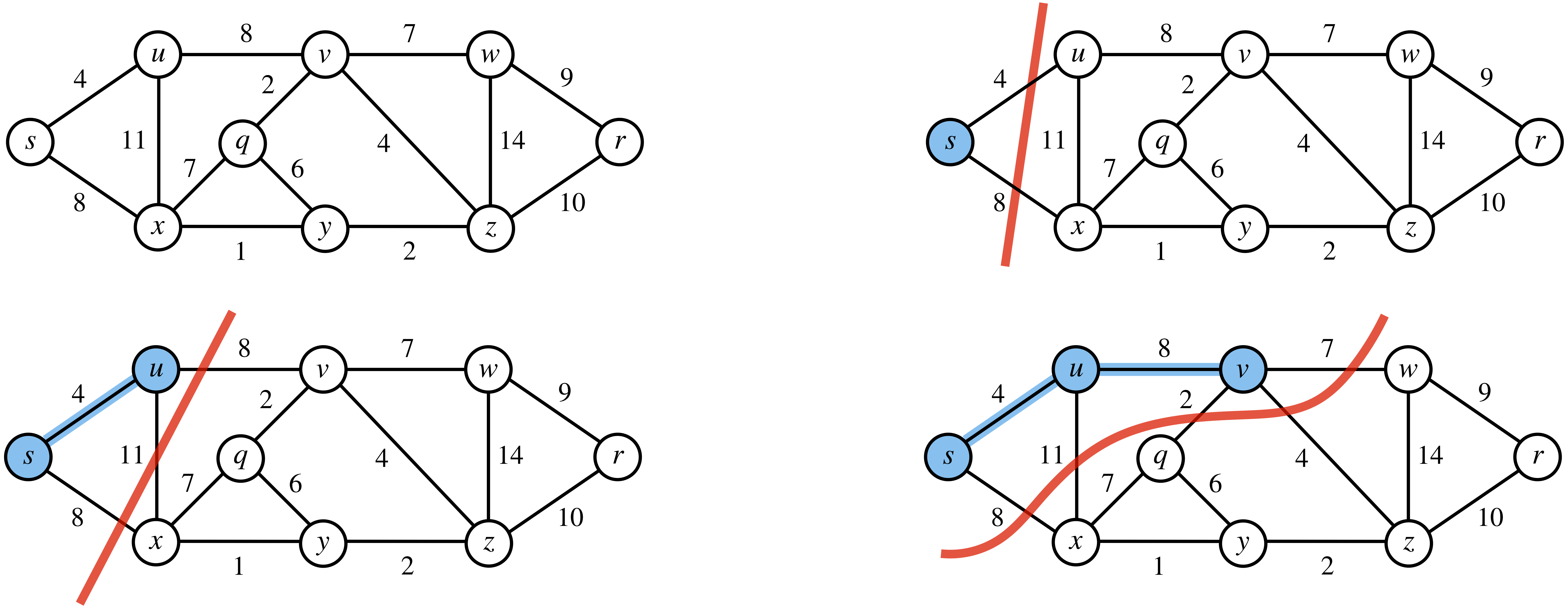




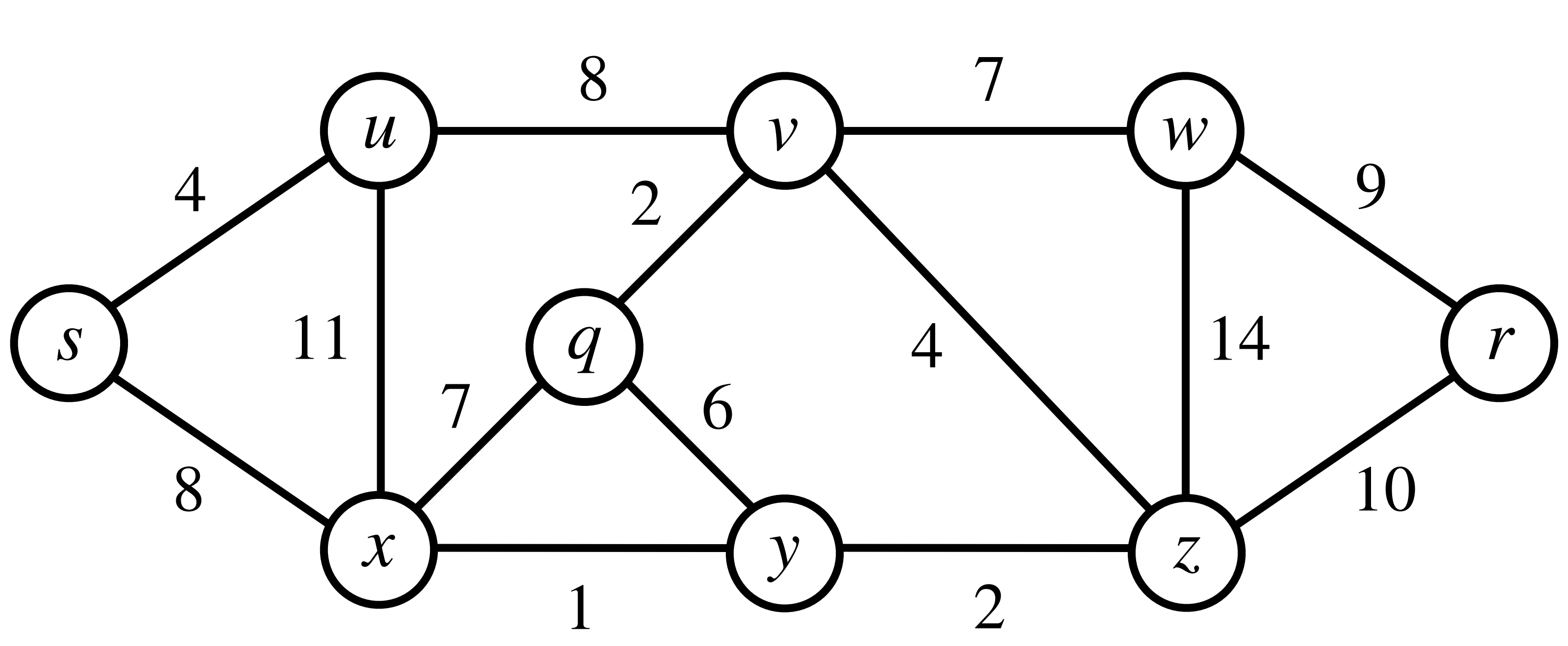
Prim's Algorithm: Demonstration

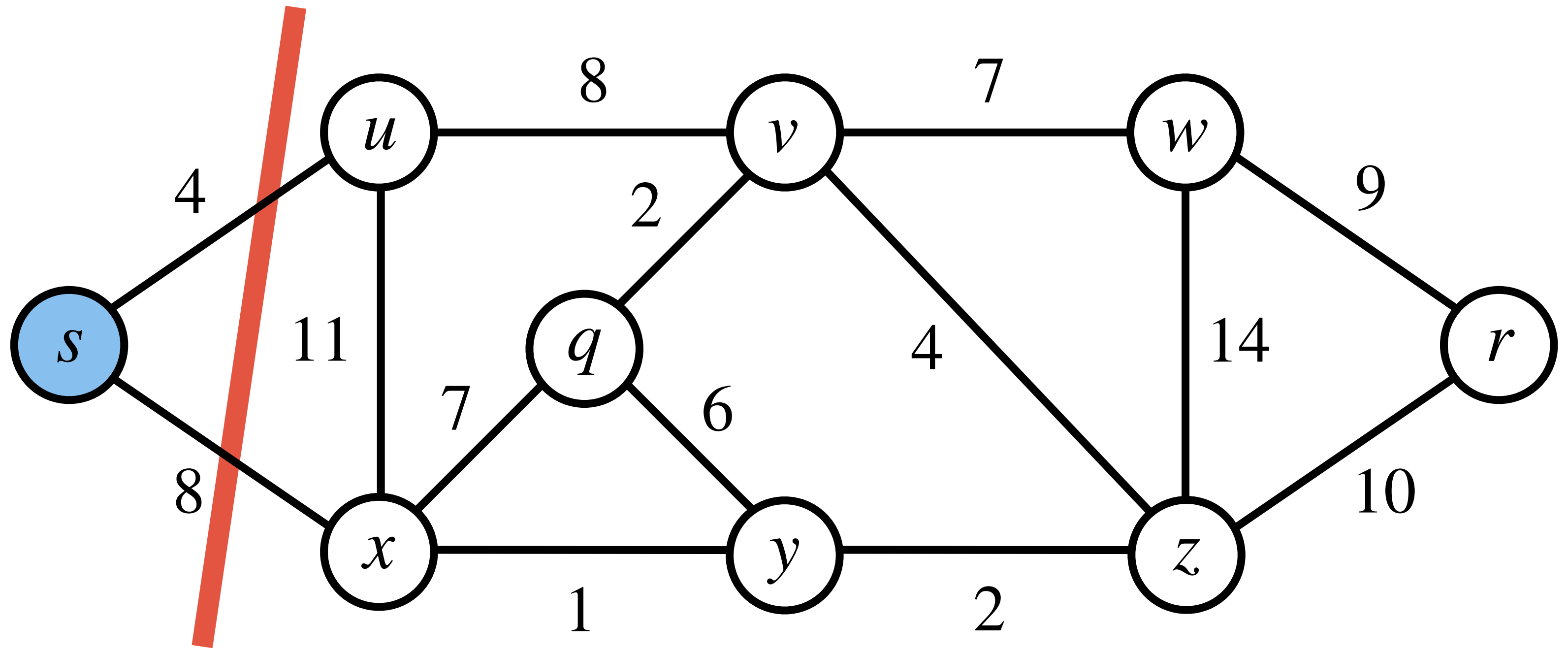


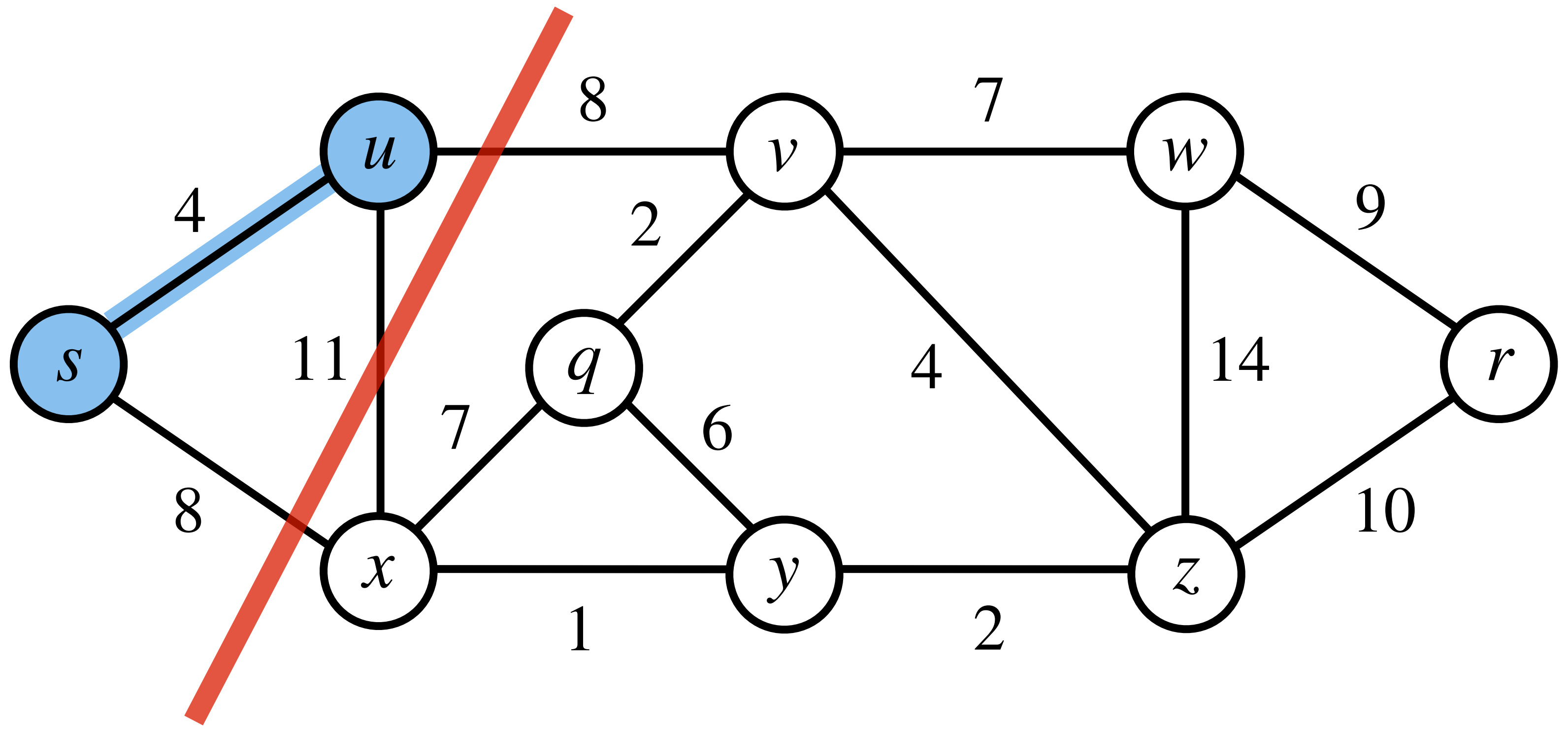
Prim's Algorithm: Demonstration

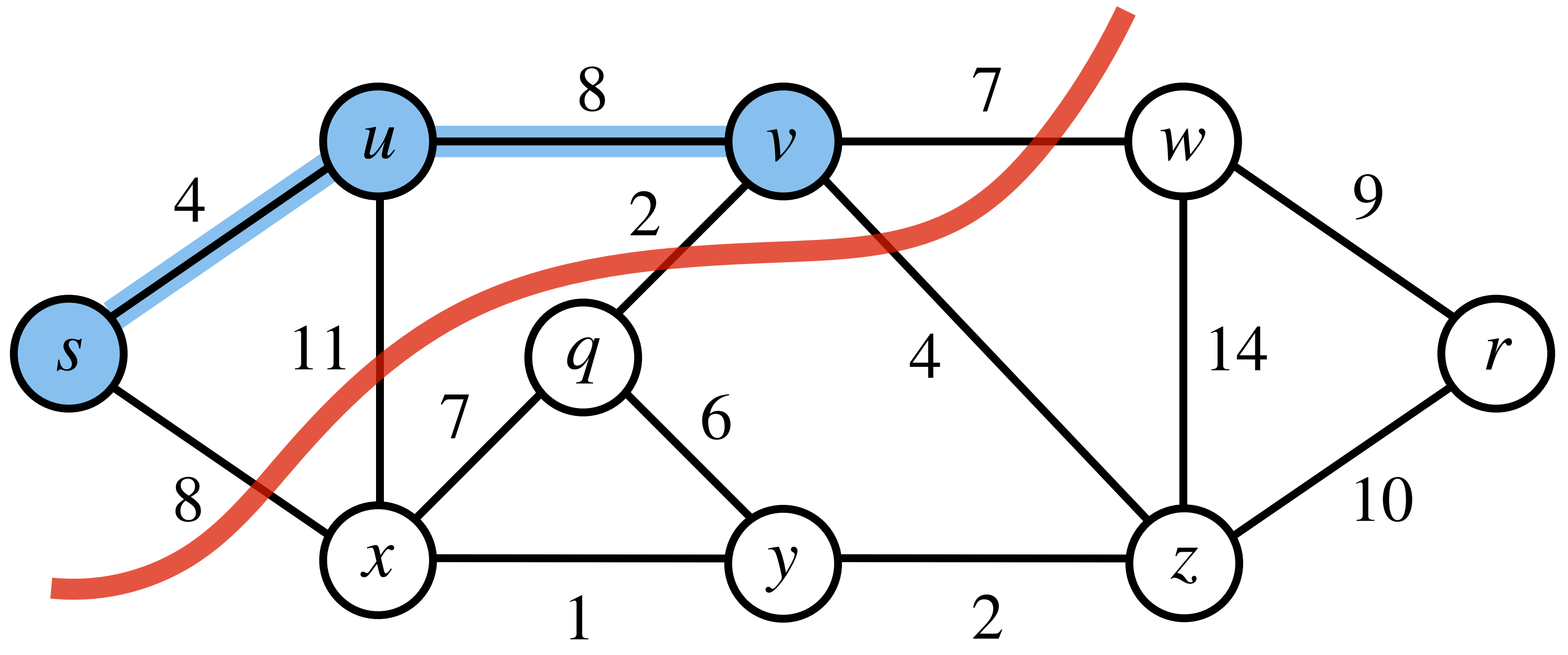


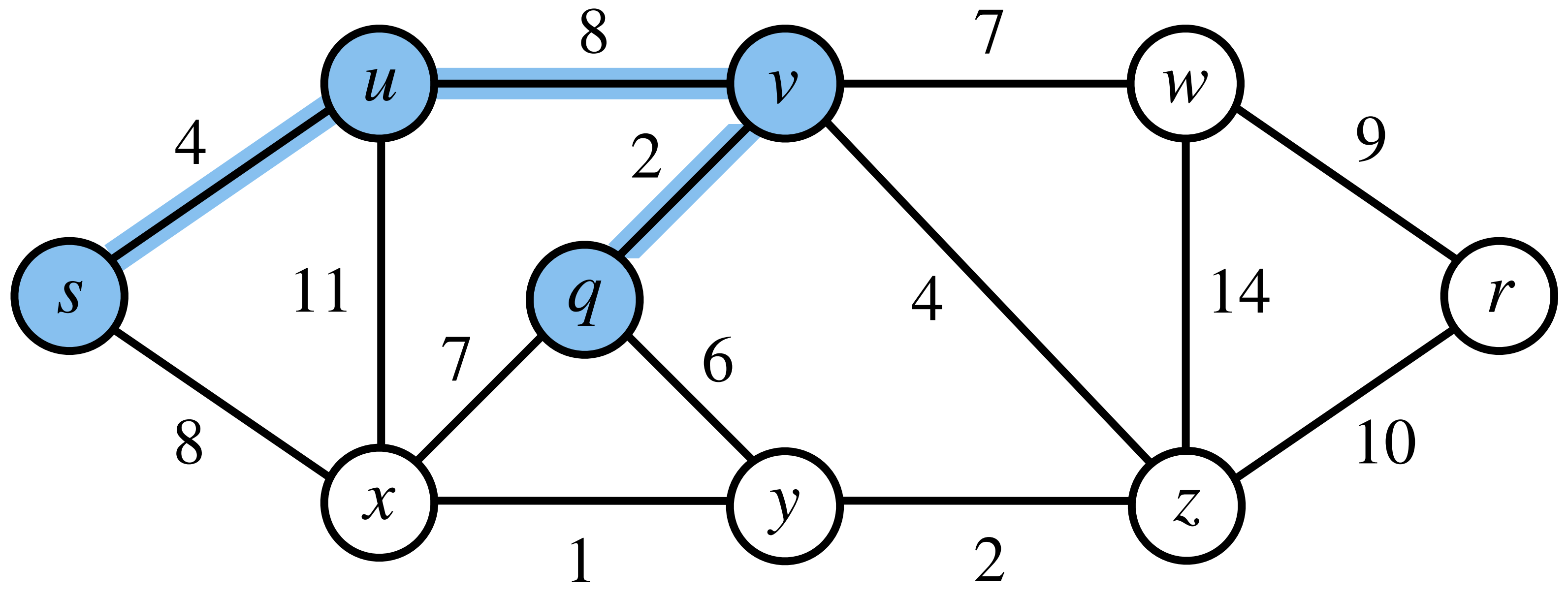
Prim’s Algorithm: Demonstration



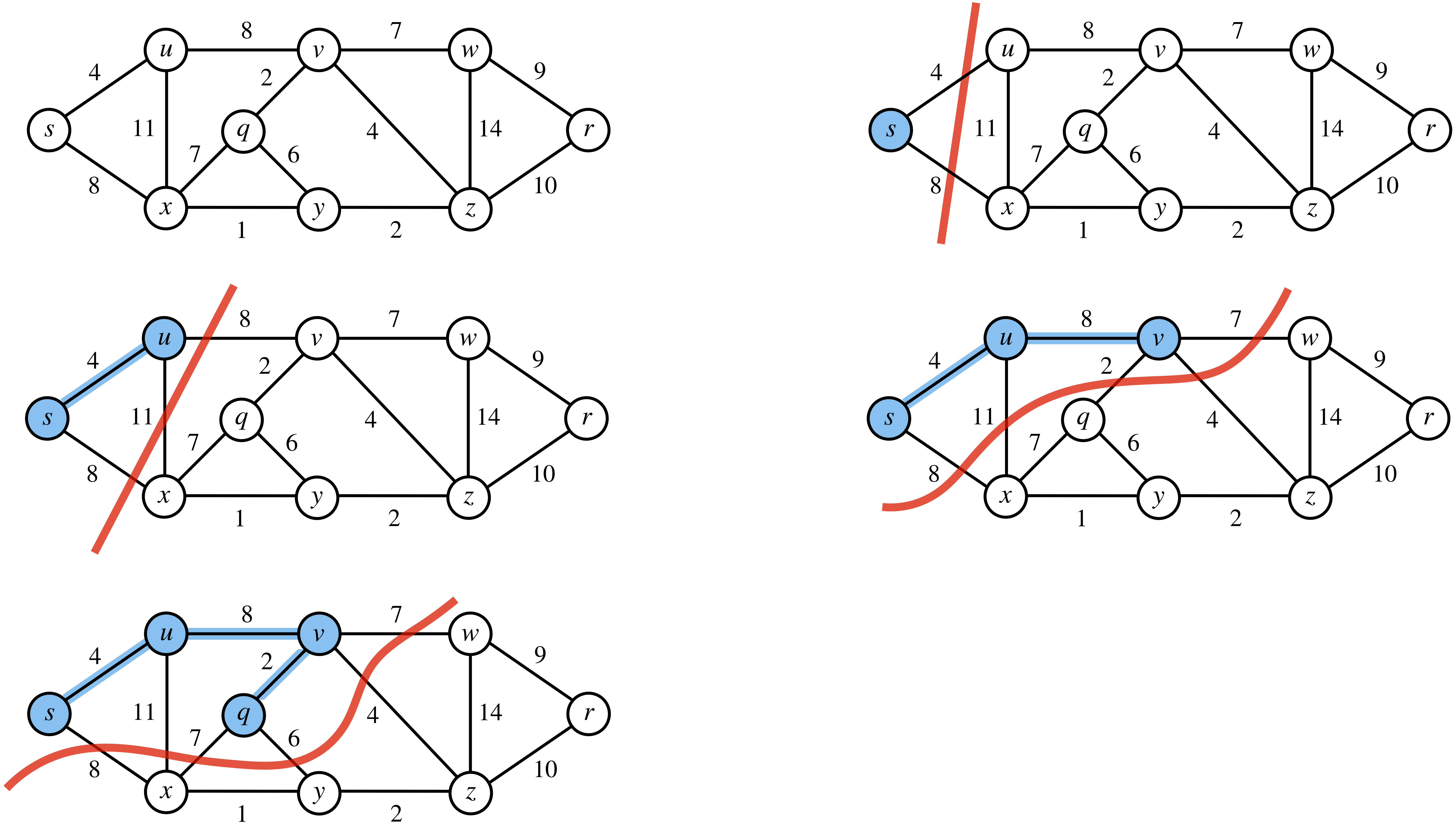




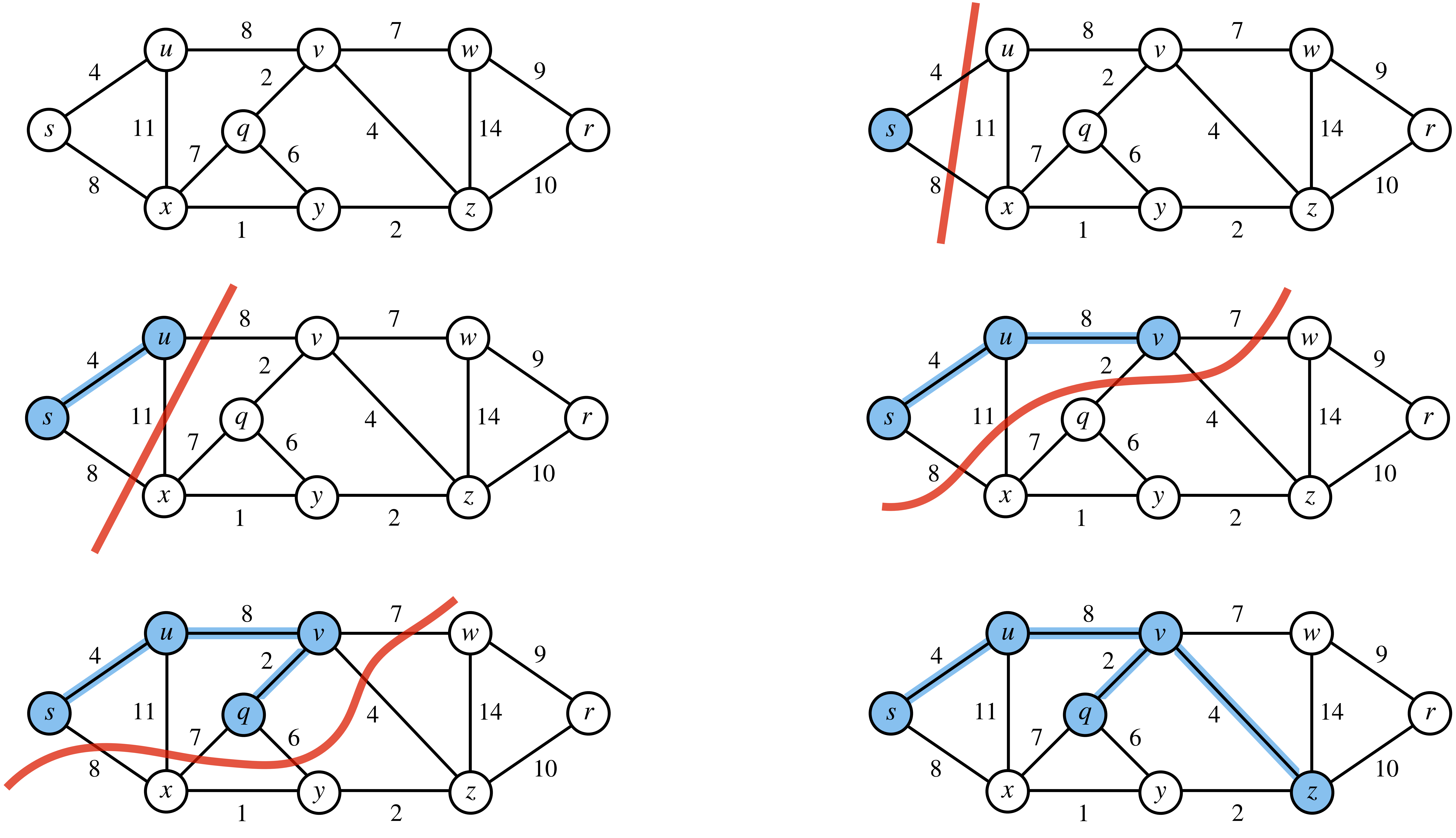




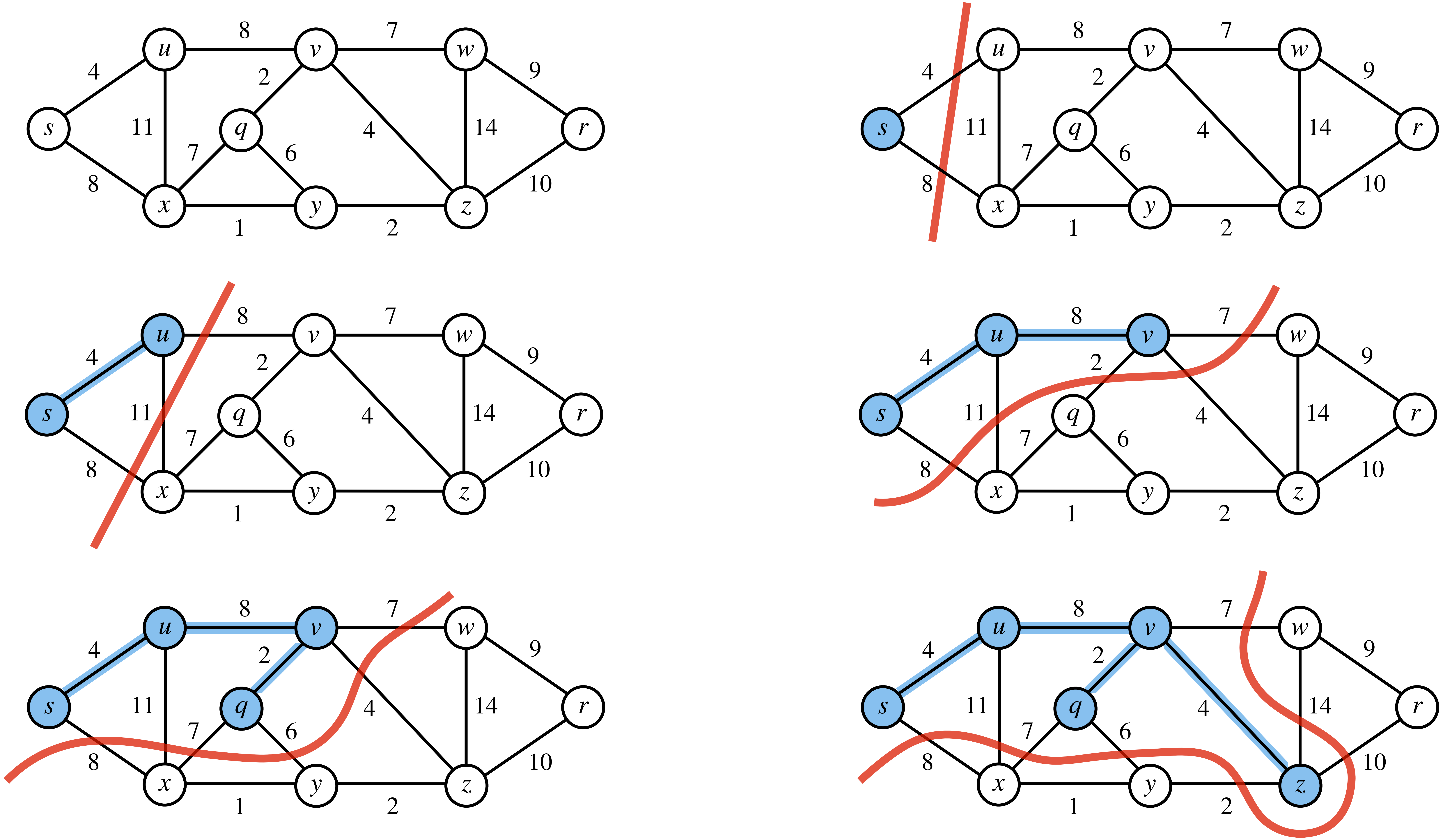
Prim's Algorithm: Demonstration



Prim’s Algorithm: Demonstration

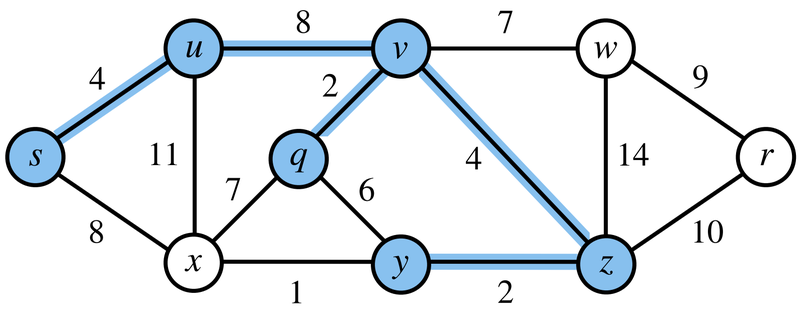


Prim’s Algorithm: Demonstration

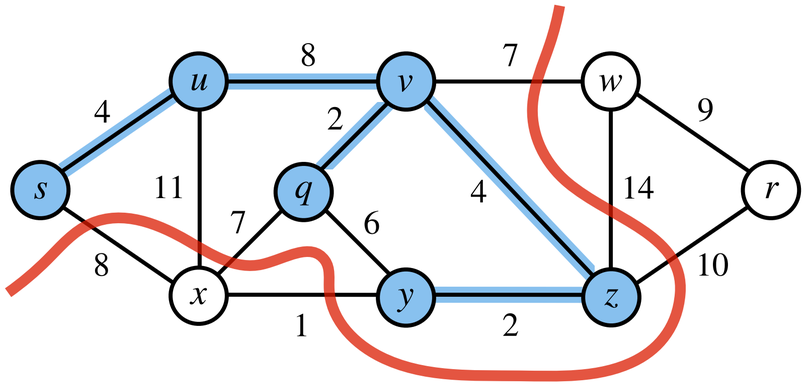


Prim’s Algorithm: Demonstration

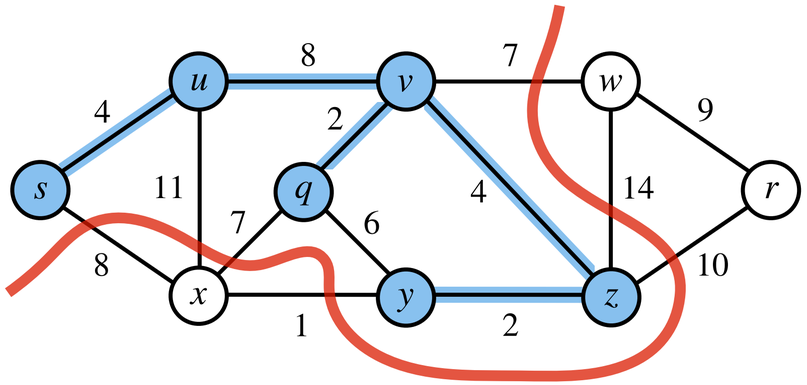
Prim's Algorithm: Demonstration

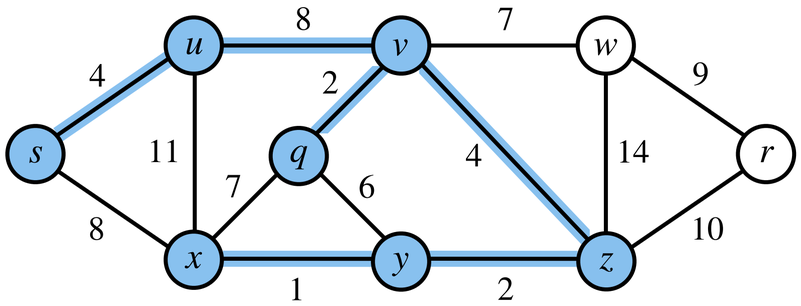


Prim's Algorithm: Demonstration

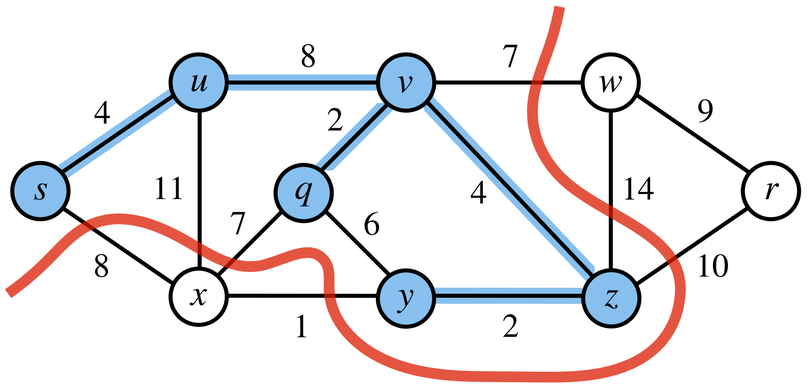


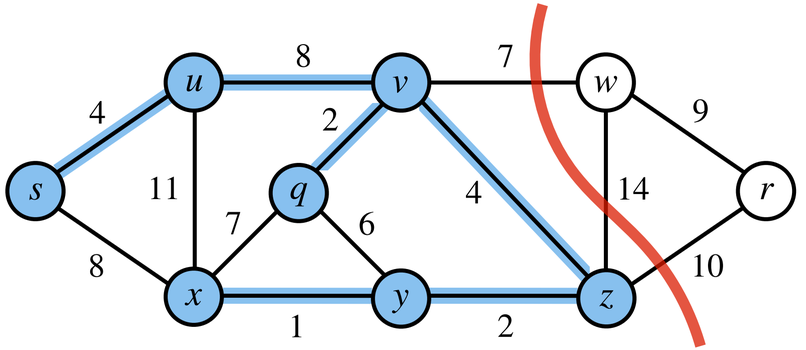
Prim's Algorithm: Demonstration



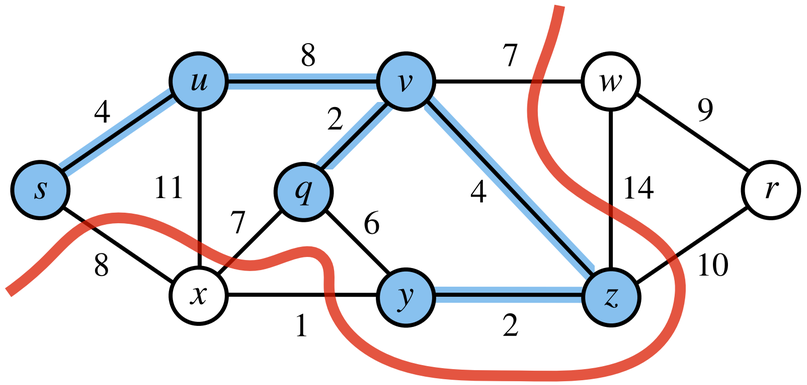


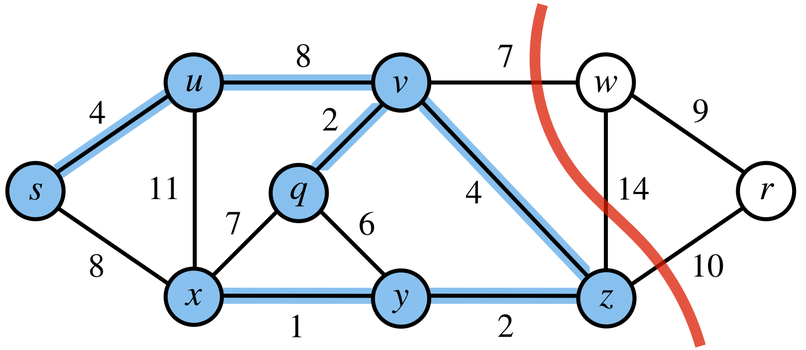
Prim's Algorithm: Demonstration

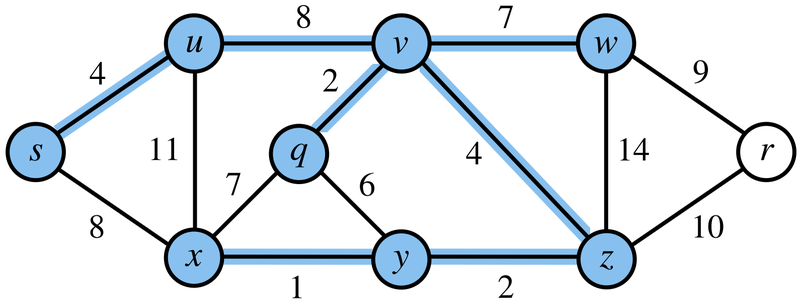




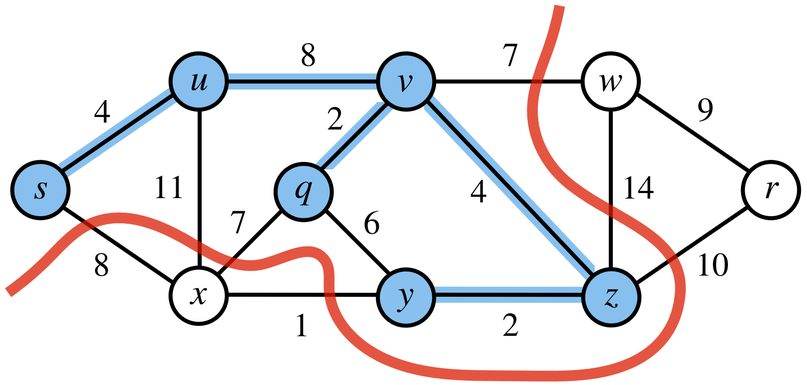
Prim's Algorithm: Demonstration

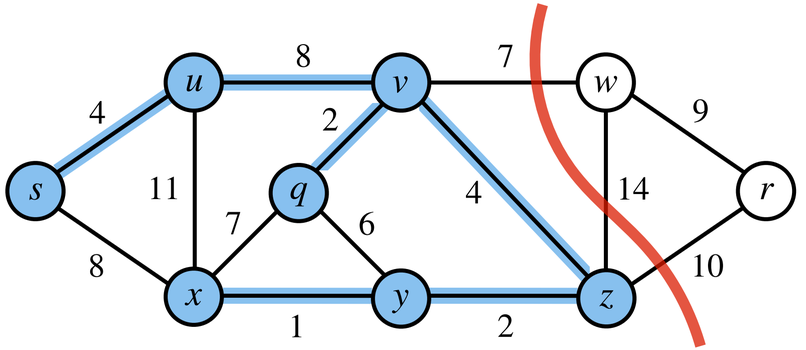


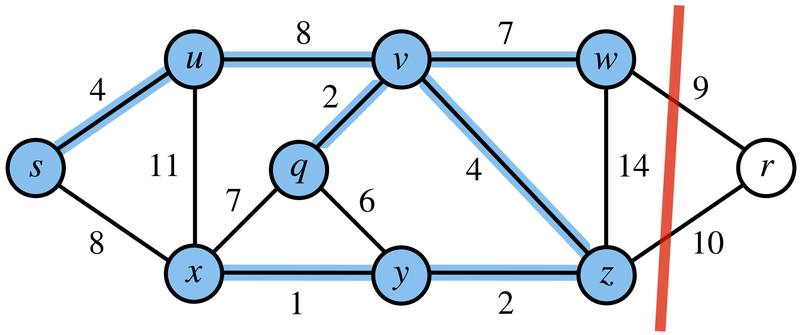




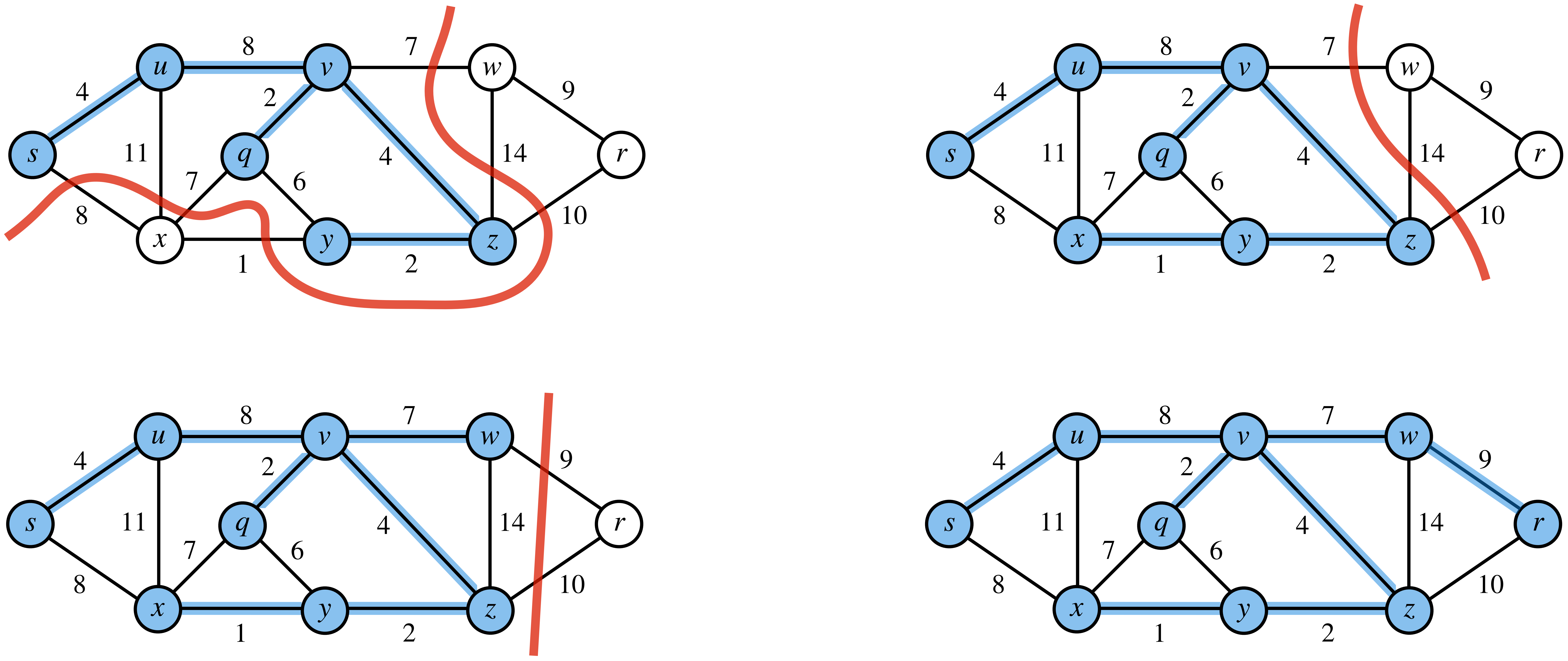
Prim's Algorithm: Demonstration





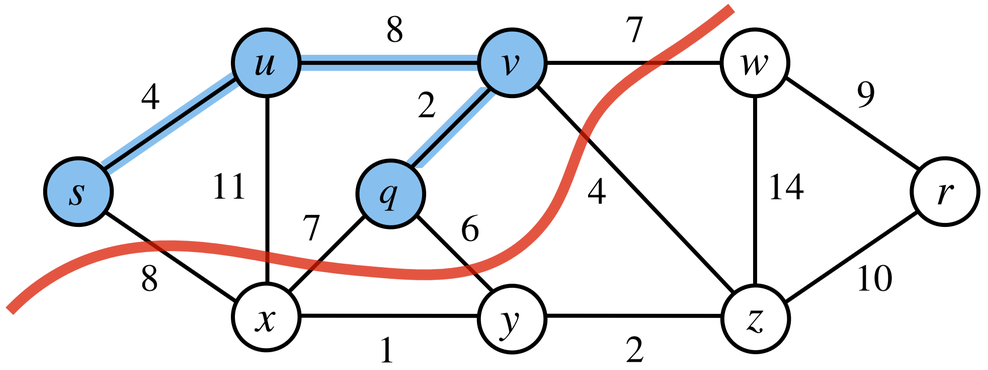


Prim's Algorithm: Demonstration

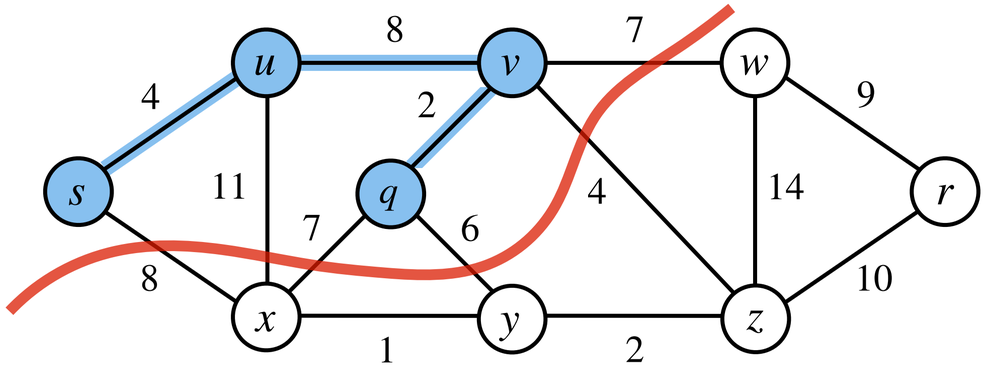


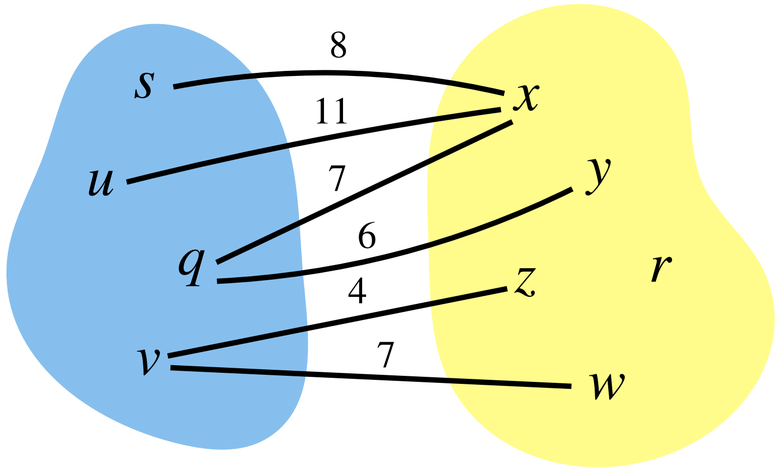
Prim’s Algorithm: Implementation

Prim's Algorithm: Implementation

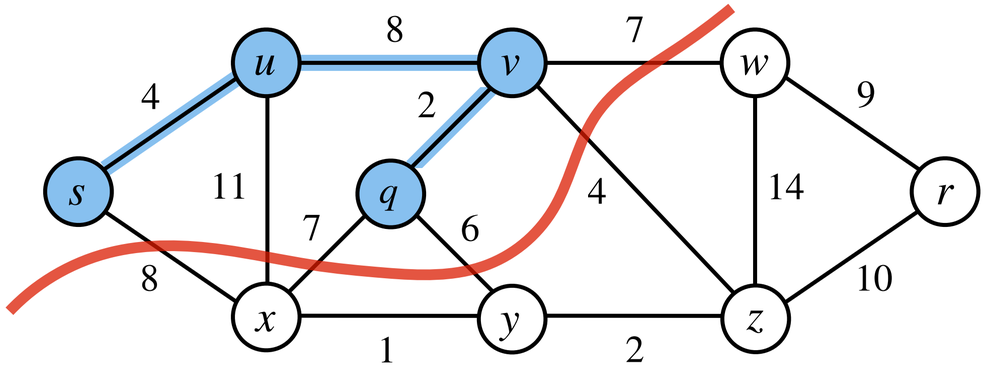


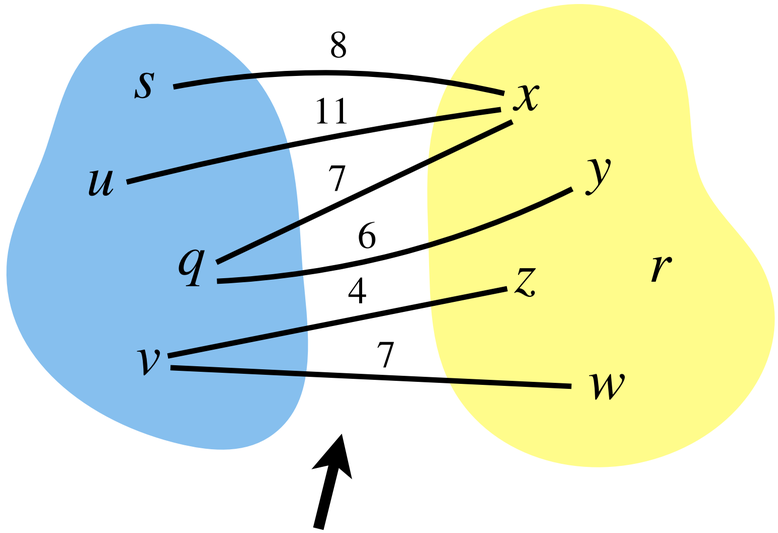
Prim's Algorithm: Implementation





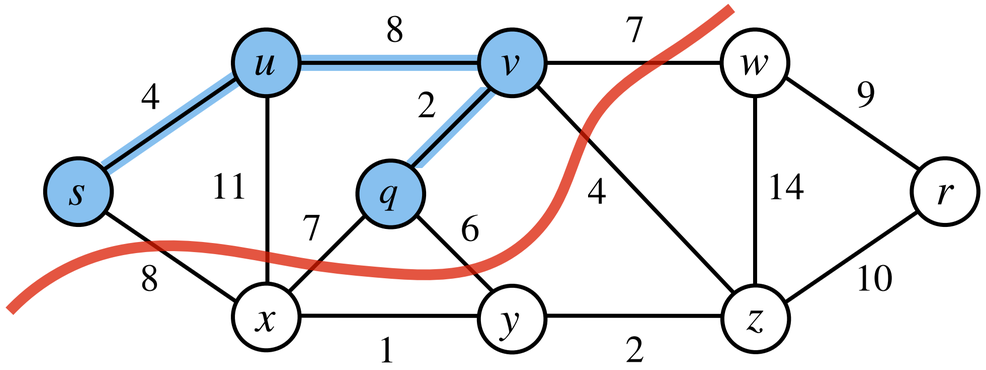
Prim's Algorithm: Implementation

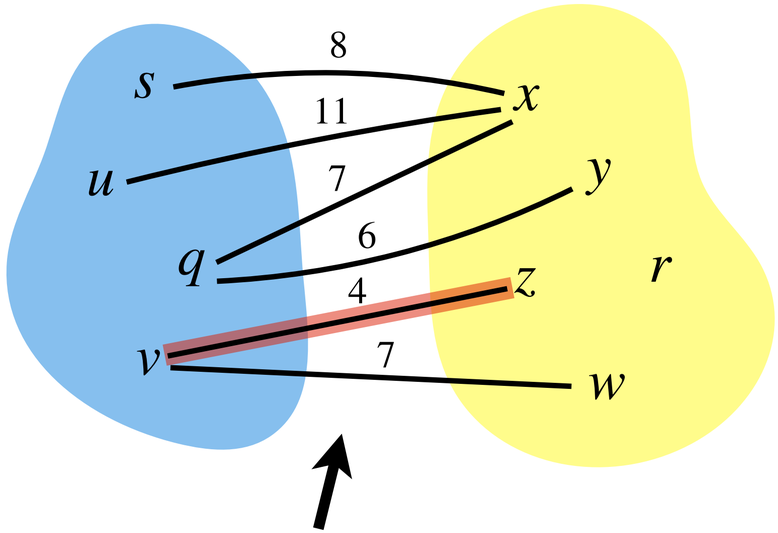




Find a least weight edge straightforwardly.

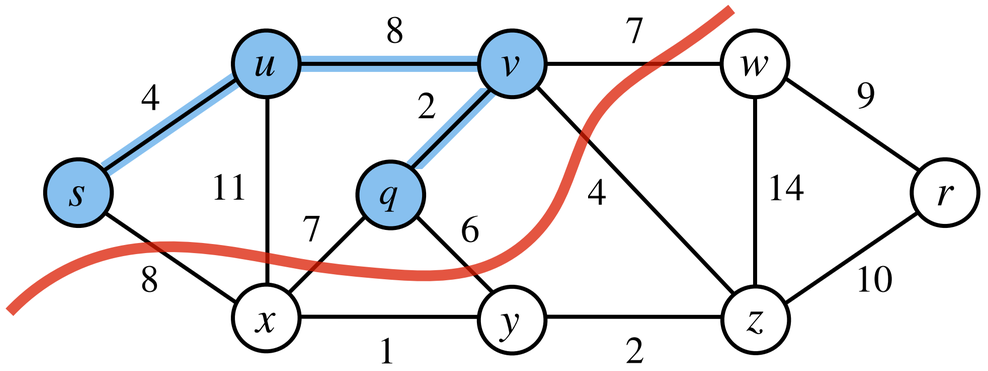
Prim's Algorithm: Implementation

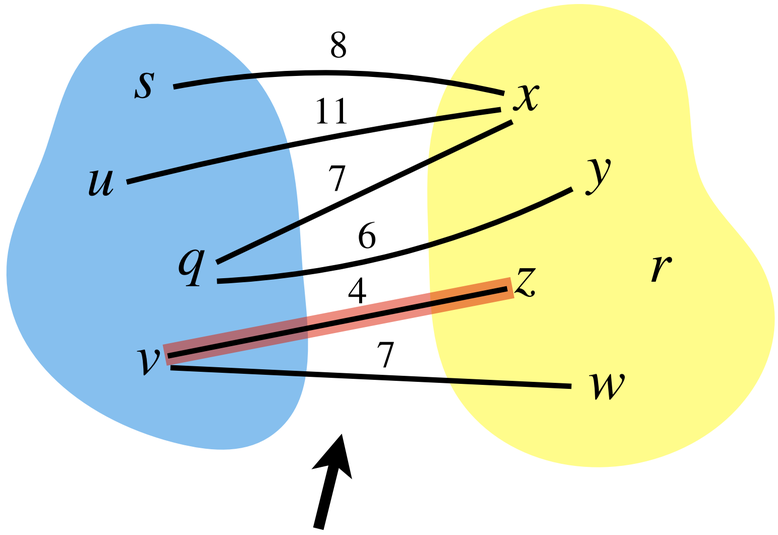


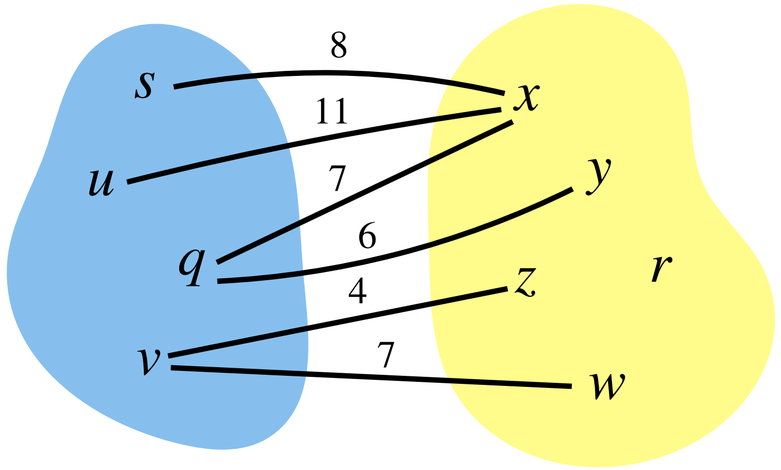


Find a least weight edge straightforwardly.

Prim's Algorithm: Implementation

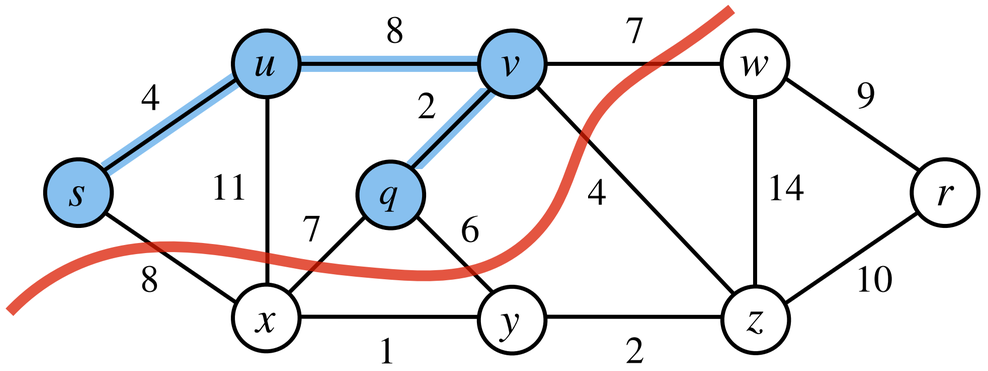


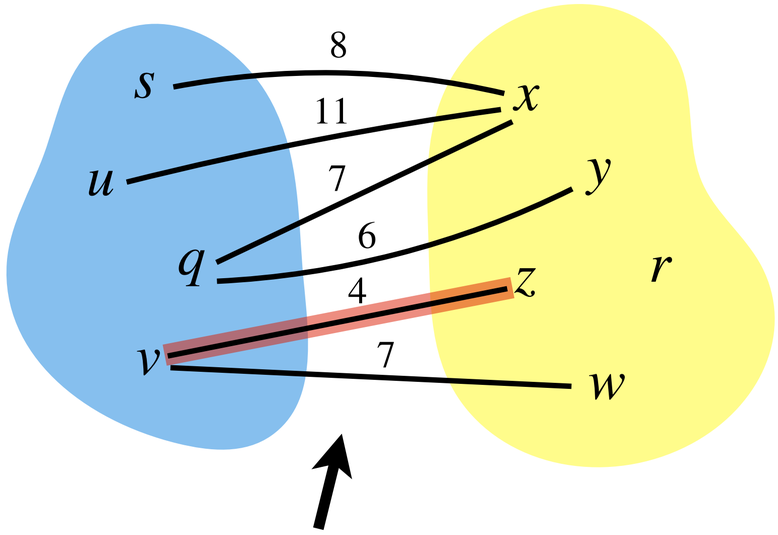




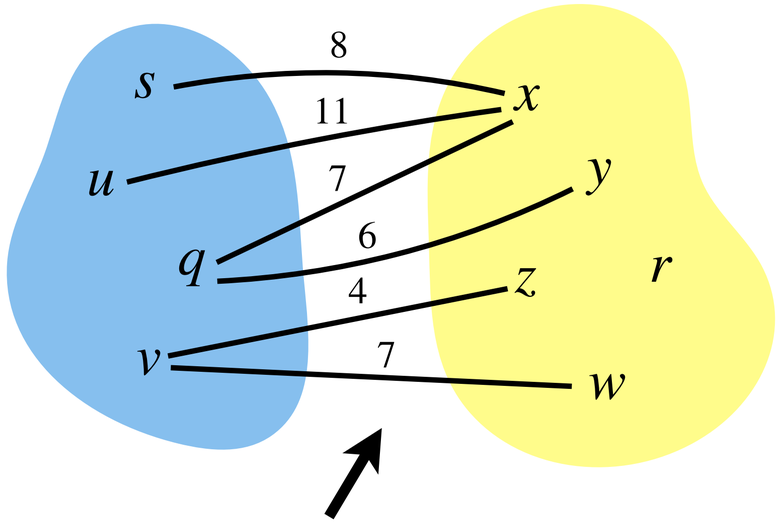
Find a least weight edge straightforwardly.

Prim's Algorithm: Implementation



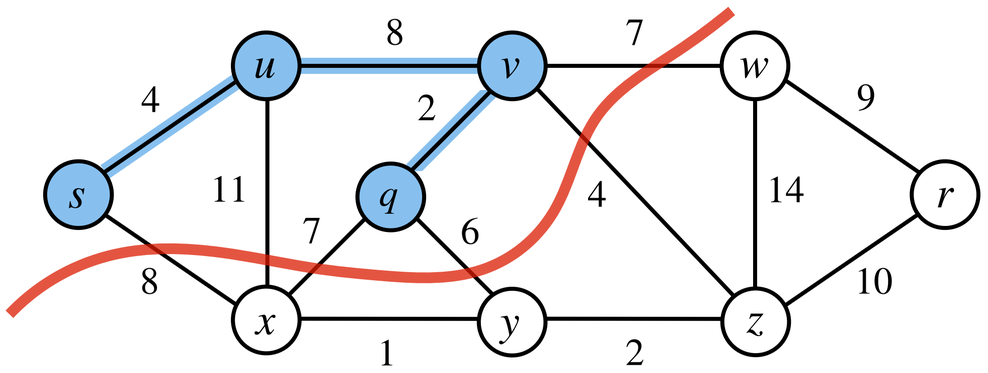


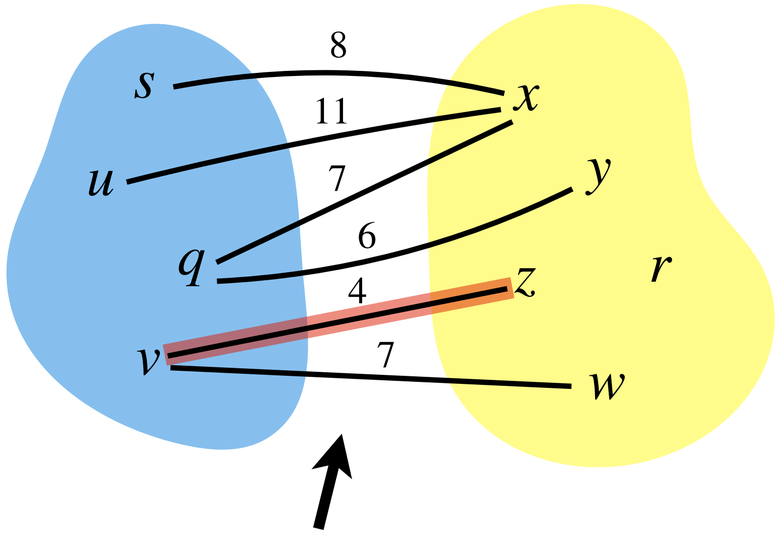
Find a least weight edge straightforwardly.



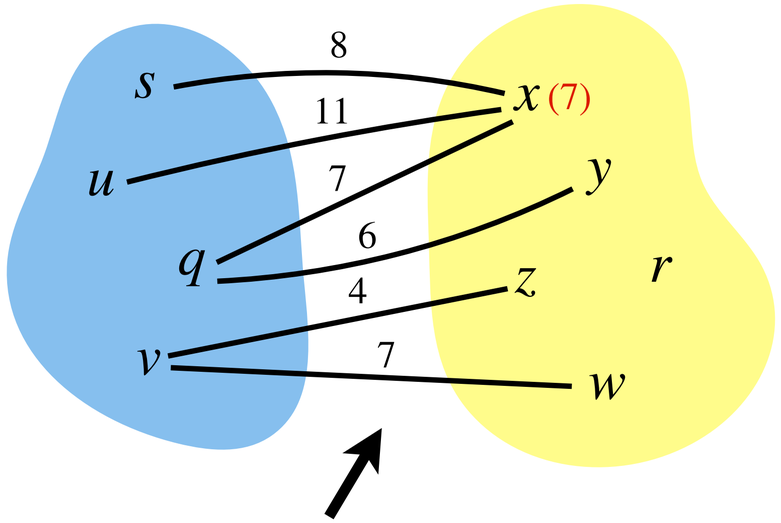
For every vertex not in tree, maintain a *key* that is equal to the weight of a least weight edge from a tree vertex to it.

Prim's Algorithm: Implementation



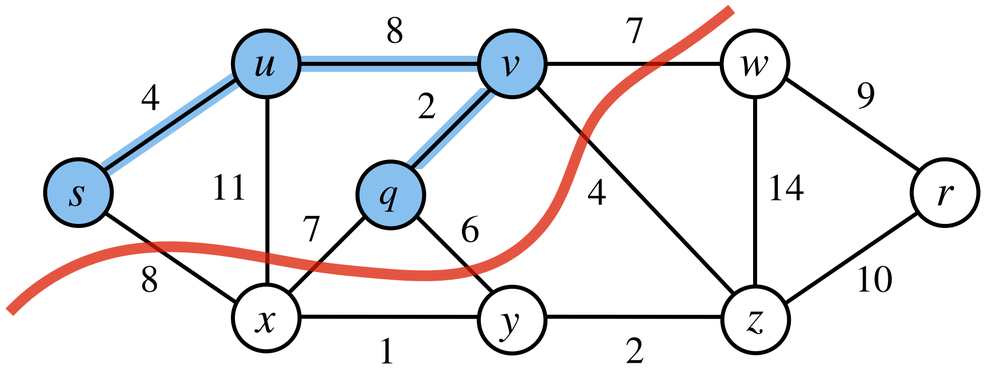


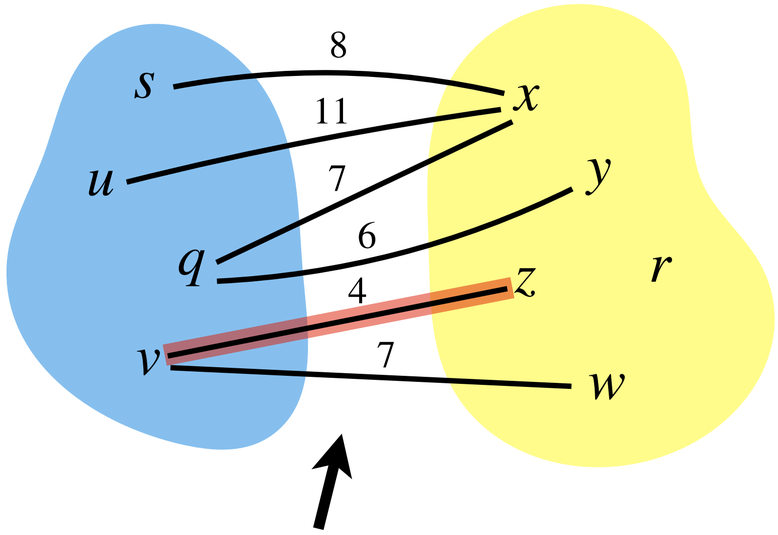
Find a least weight edge straightforwardly.



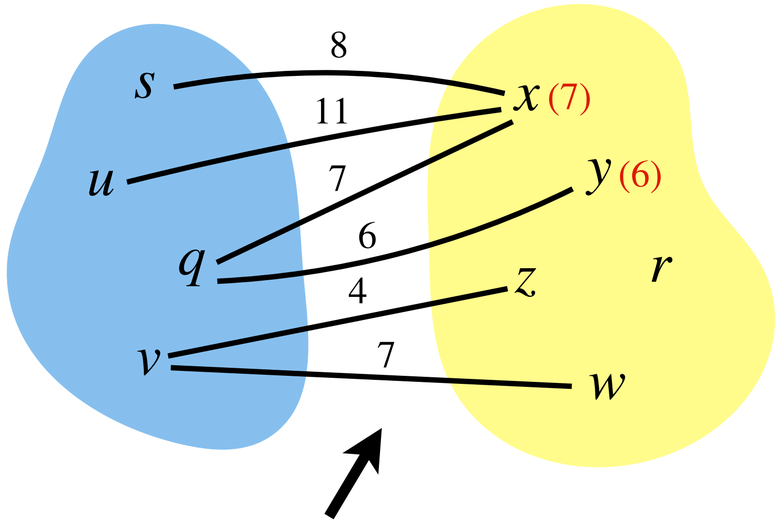
For every vertex not in tree, maintain a *key* that is equal to the weight of a least weight edge from a tree vertex to it.

Prim's Algorithm: Implementation



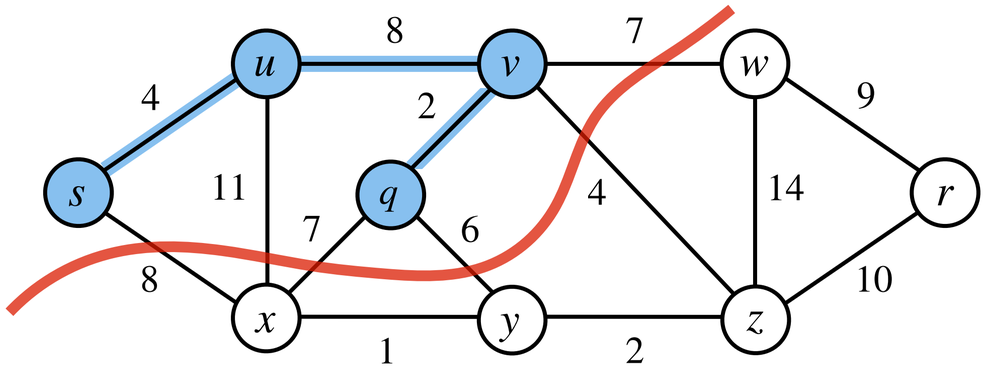


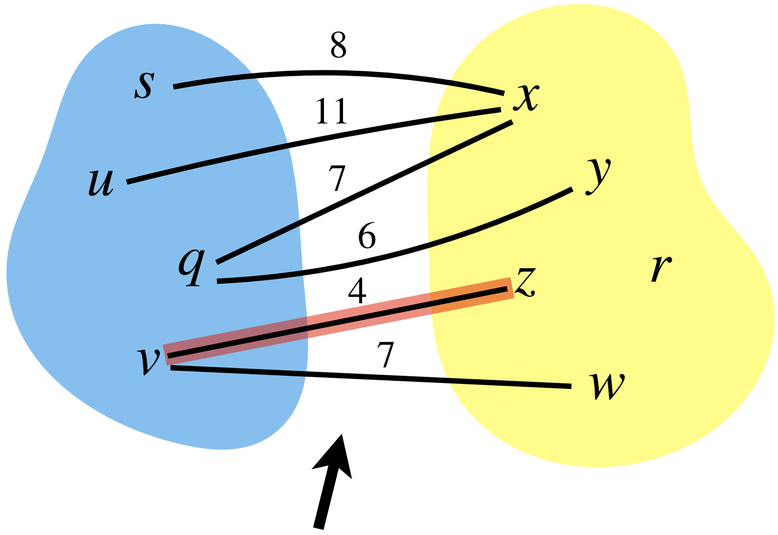
Find a least weight edge straightforwardly.



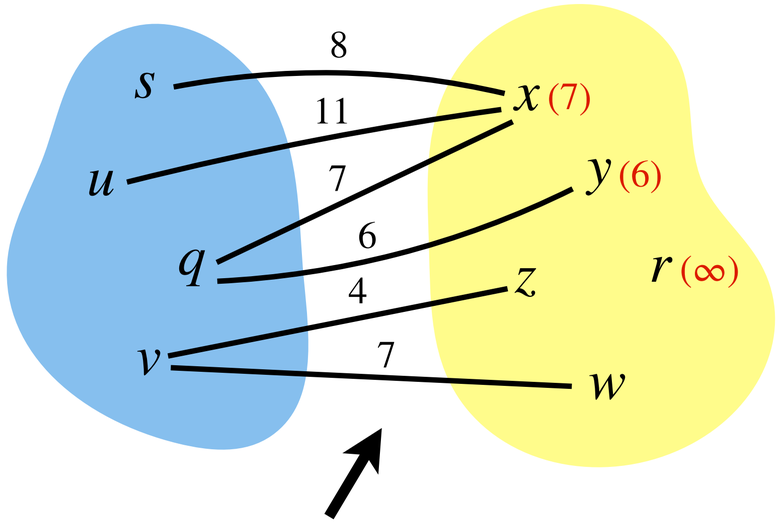
For every vertex not in tree, maintain a *key* that is equal to the weight of a least weight edge from a tree vertex to it.

Prim's Algorithm: Implementation



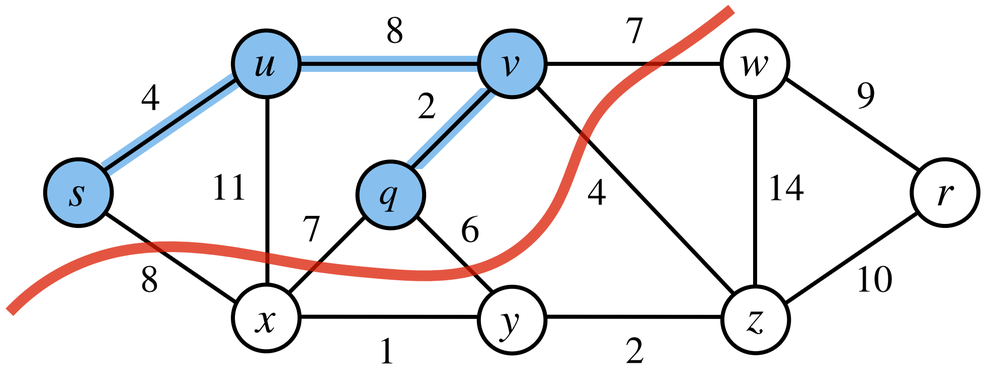


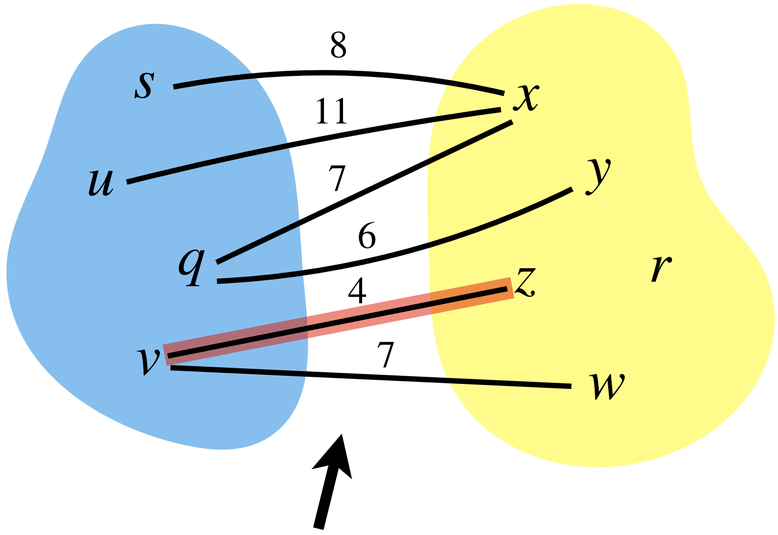
Find a least weight edge straightforwardly.



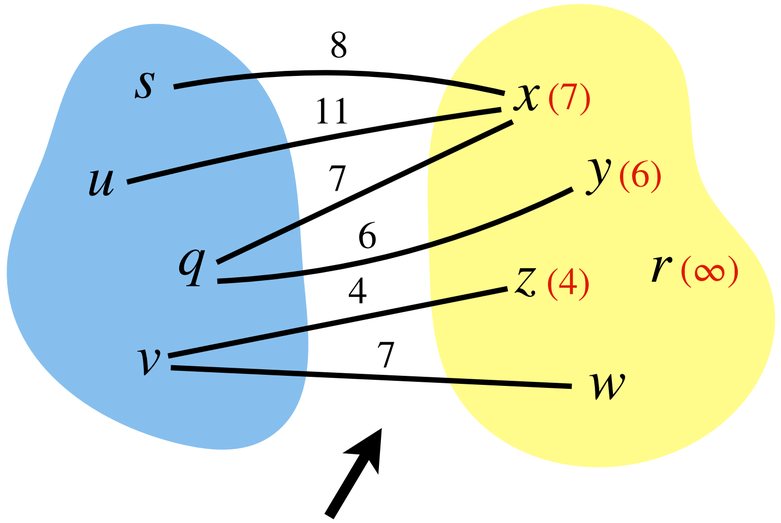
For every vertex not in tree, maintain a *key* that is equal to the weight of a least weight edge from a tree vertex to it.

Prim's Algorithm: Implementation



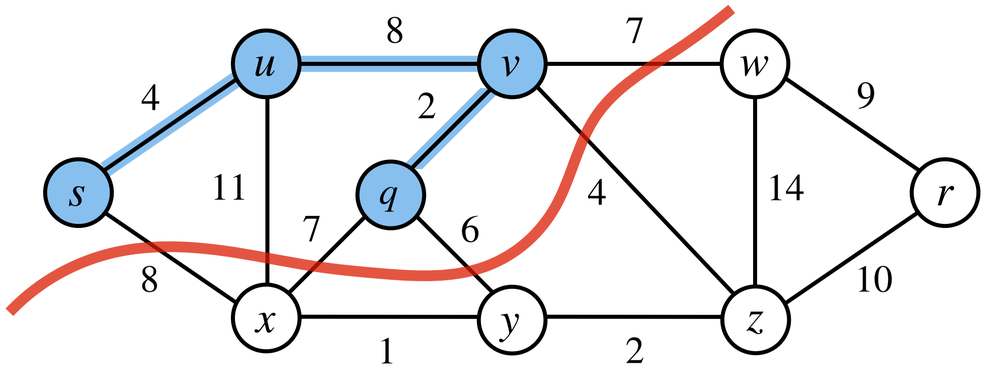


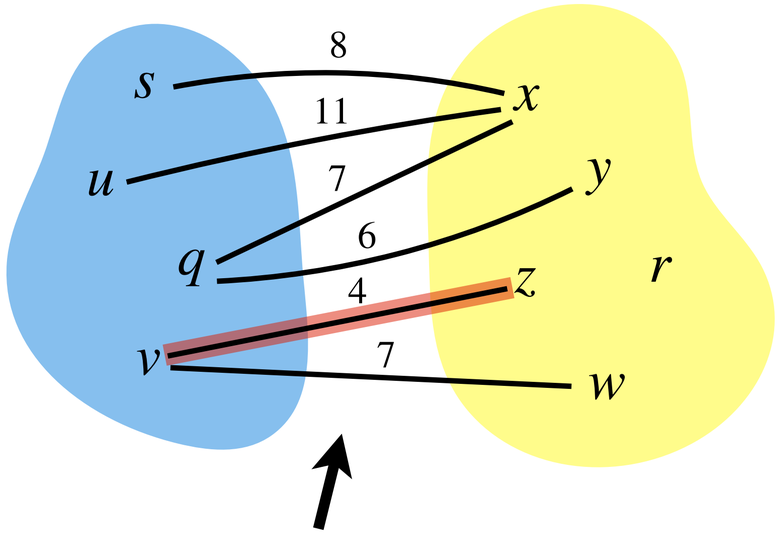
Find a least weight edge straightforwardly.



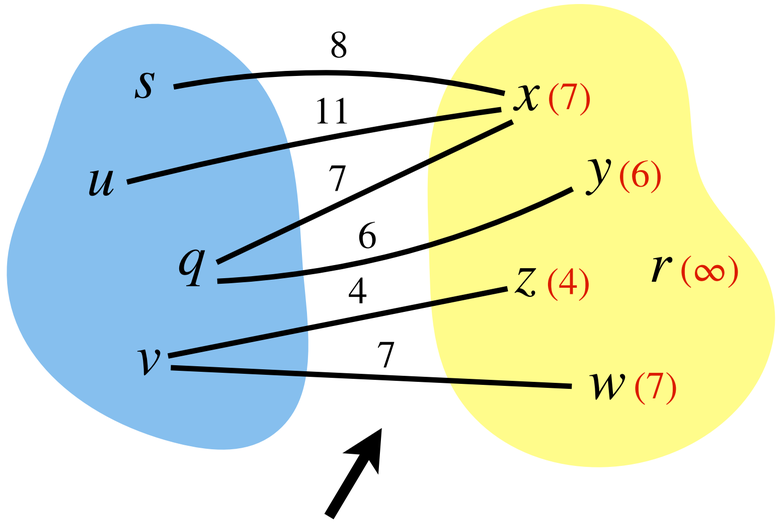
For every vertex not in tree, maintain a *key* that is equal to the weight of a least weight edge from a tree vertex to it.

Prim's Algorithm: Implementation



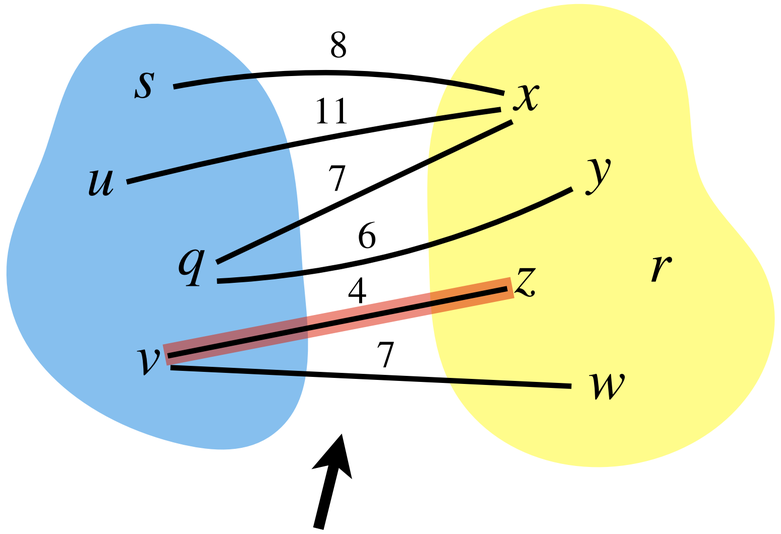


Find a least weight edge straightforwardly.

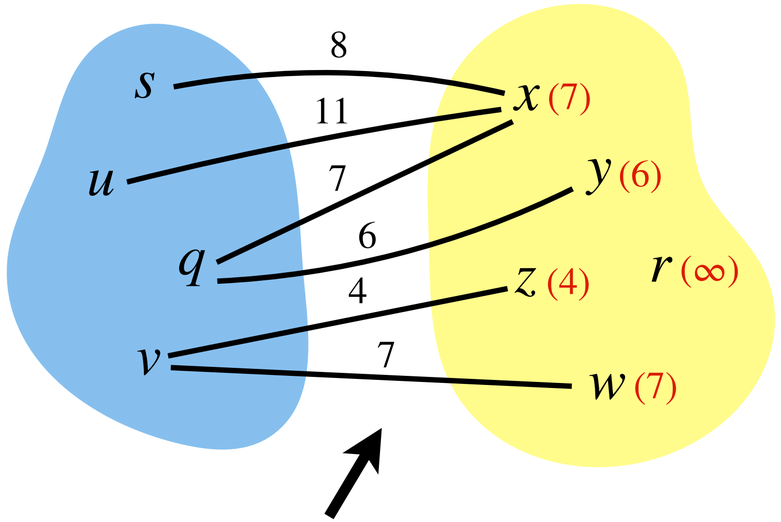


For every vertex not in tree, maintain a *key* that is equal to the weight of a least weight edge from a tree vertex to it.

Prim's Algorithm: Implementation

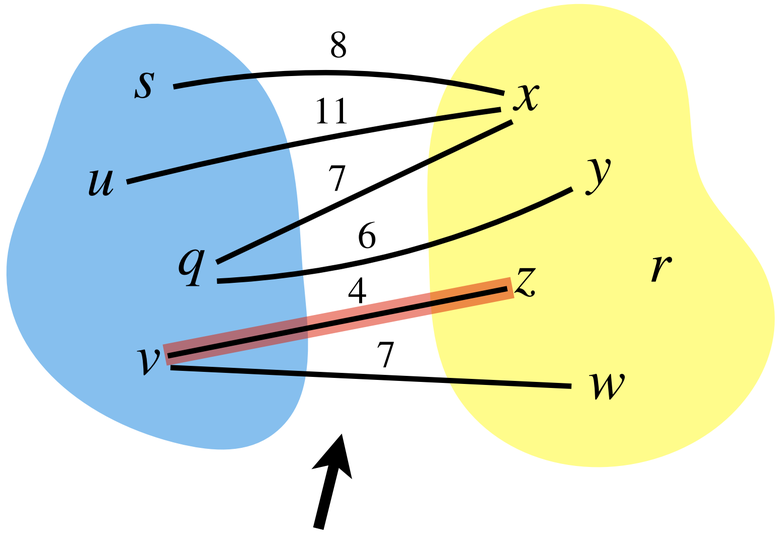


Find a least weight edge straightforwardly.

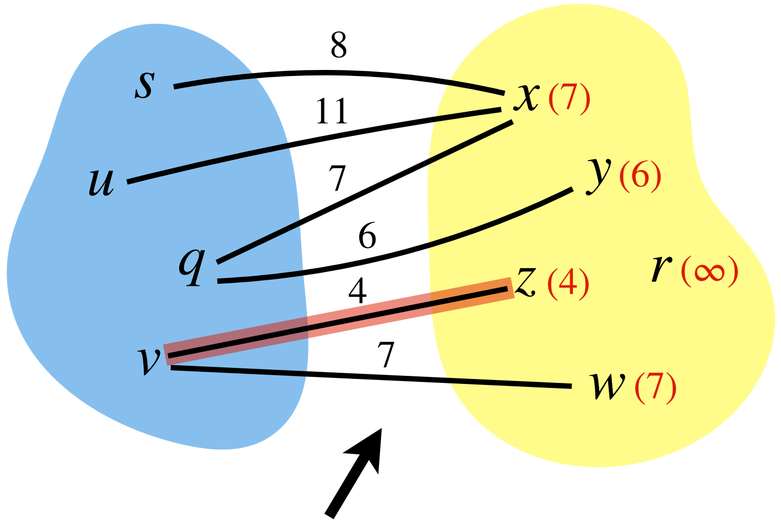


For every vertex not in tree, maintain a *key* that is equal to the weight of a least weight edge from a tree vertex to it.

Prim's Algorithm: Implementation

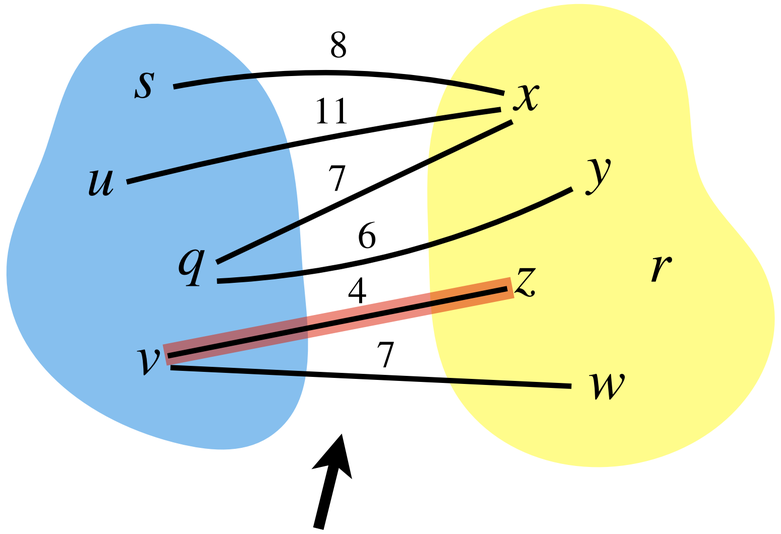


Find a least weight edge straightforwardly.

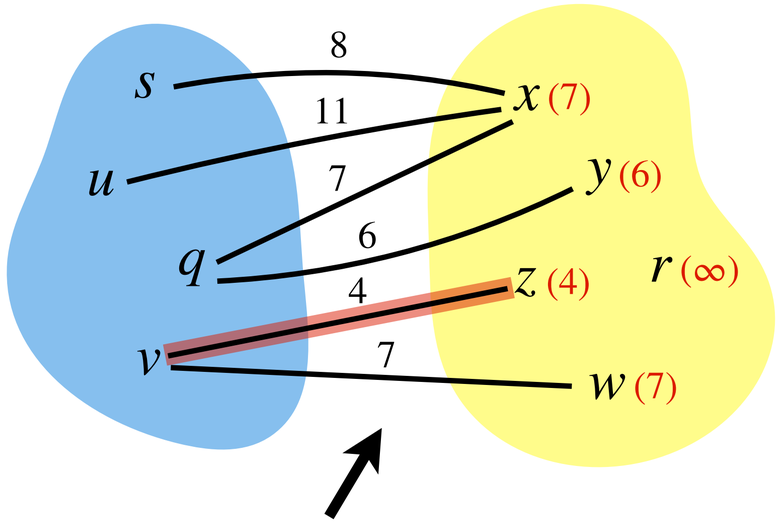


For every vertex not in tree, maintain a *key* that is equal to the weight of a least weight edge from a tree vertex to it.

Prim's Algorithm: Implementation



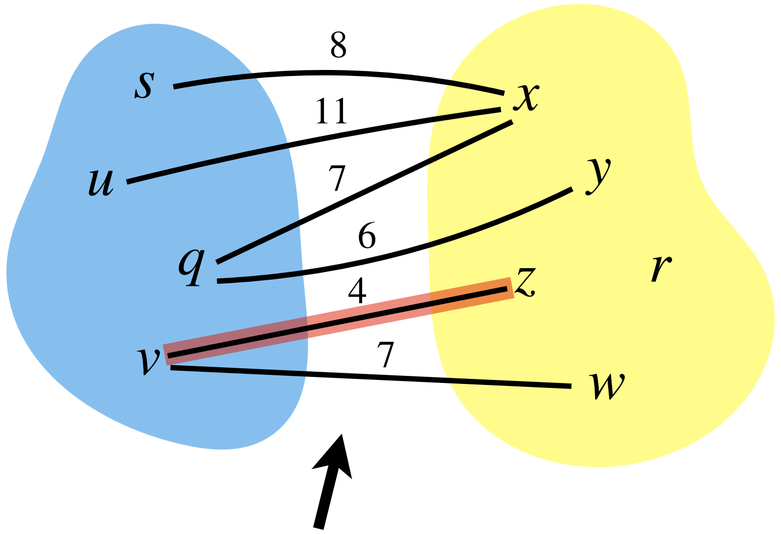
Find a least weight edge straightforwardly.



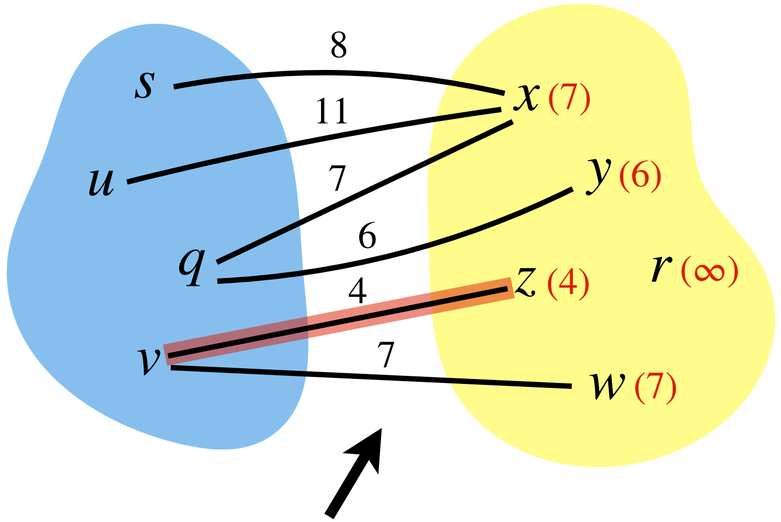
For every vertex not in tree, maintain a *key* that is equal to the weight of a least weight edge from a tree vertex to it.

Observation: Finding a least weight edge in the cut-set is the same as finding a non-tree

Prim's Algorithm: Implementation



Find a least weight edge straightforwardly.

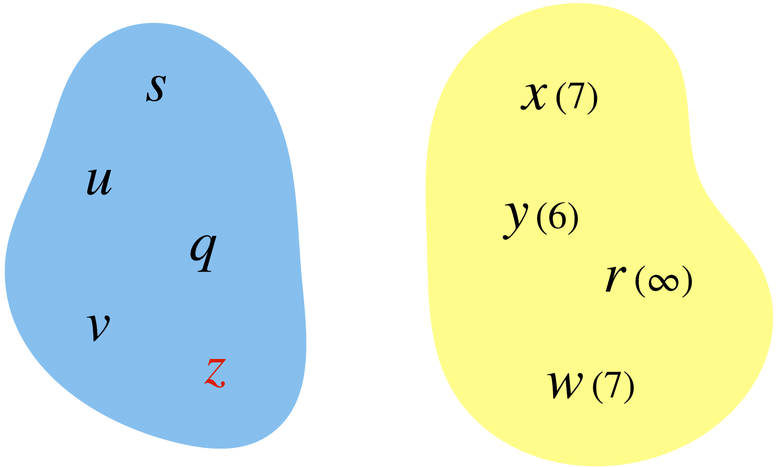


For every vertex not in tree, maintain a *key* that is equal to the weight of a least weight edge from a tree vertex to it.

Observation: Finding a least weight edge in the cut-set is the same as finding a non-tree vertex with minimum *key* and the edge which connects it to tree.

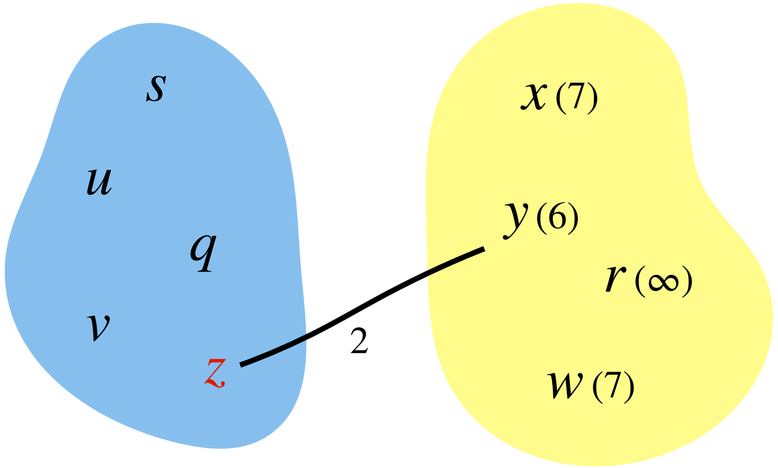
Prim's Algorithm: Implementation

Updating *keys*:



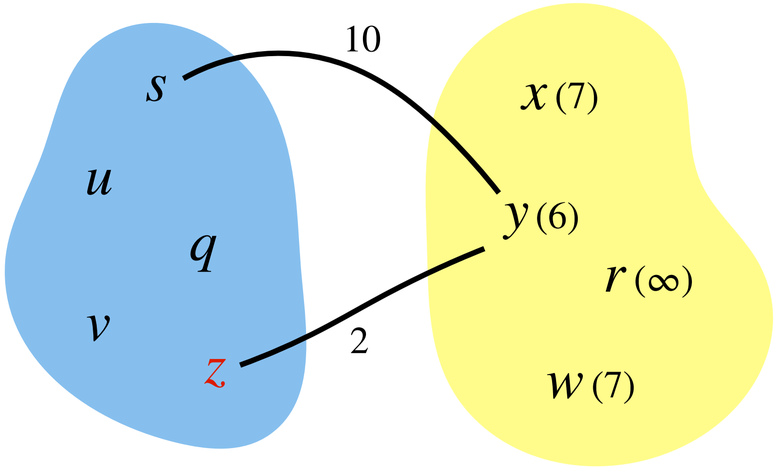
Prim's Algorithm: Implementation

Updating *keys*:



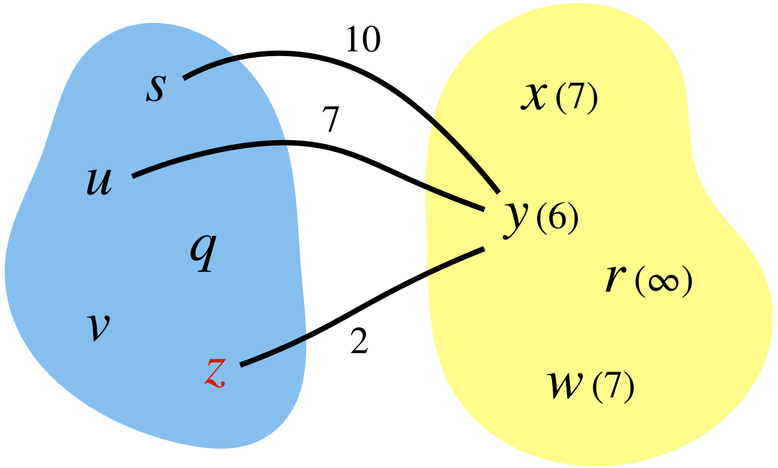
Prim's Algorithm: Implementation

Updating *keys*:



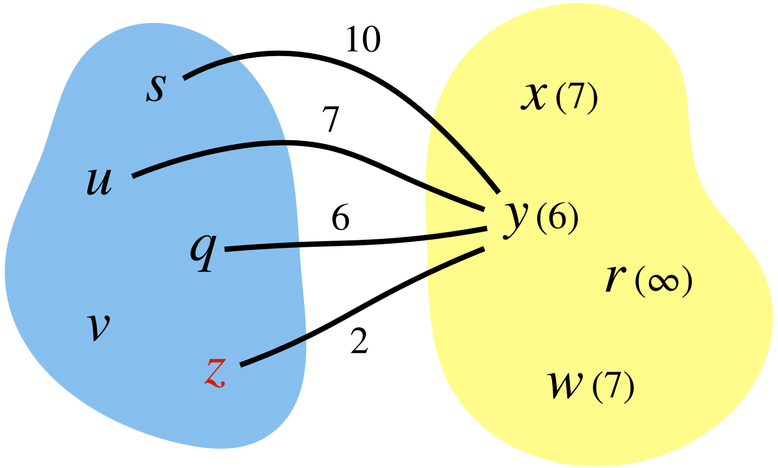
Prim's Algorithm: Implementation

Updating *keys*:



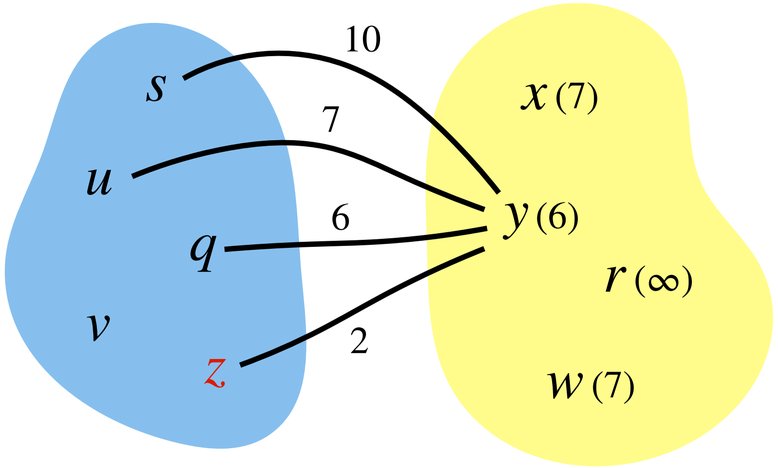
Prim's Algorithm: Implementation

Updating *keys*:



Prim's Algorithm: Implementation

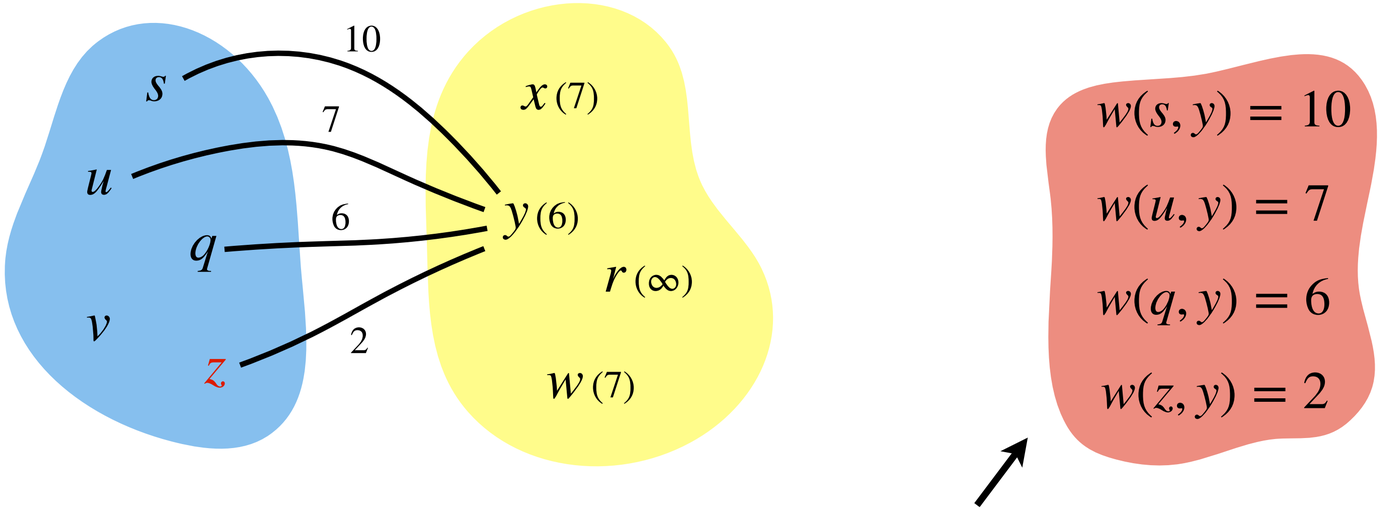
Updating *keys*:



key[*y*] will be minimum of these values.

Prim's Algorithm: Implementation

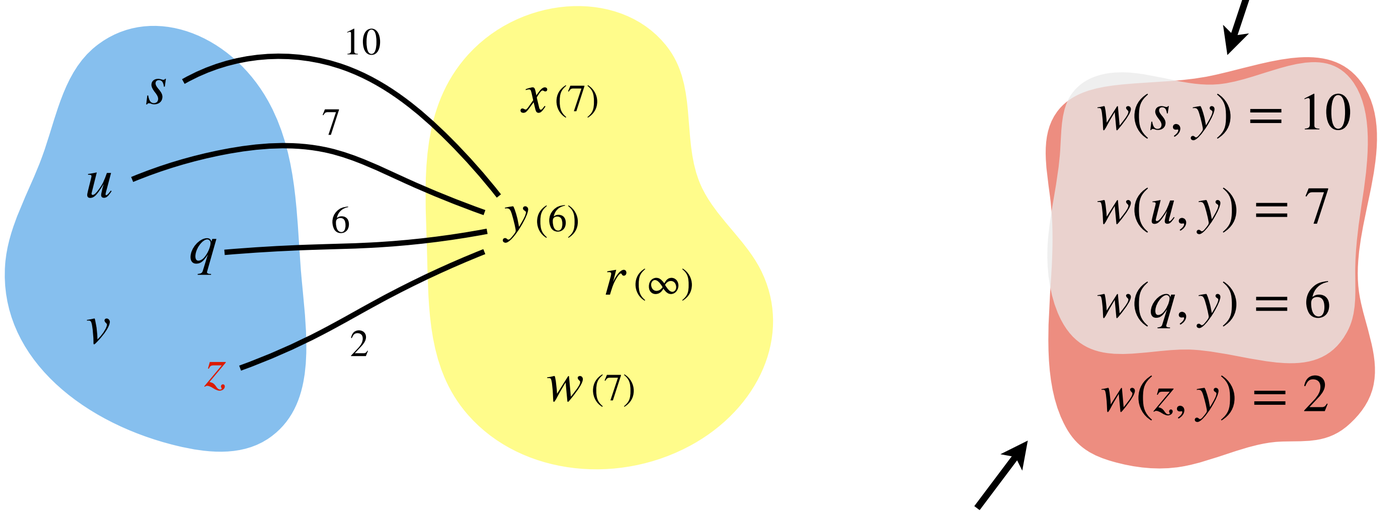
Updating *keys*:



key[*y*] will be minimum of these values.

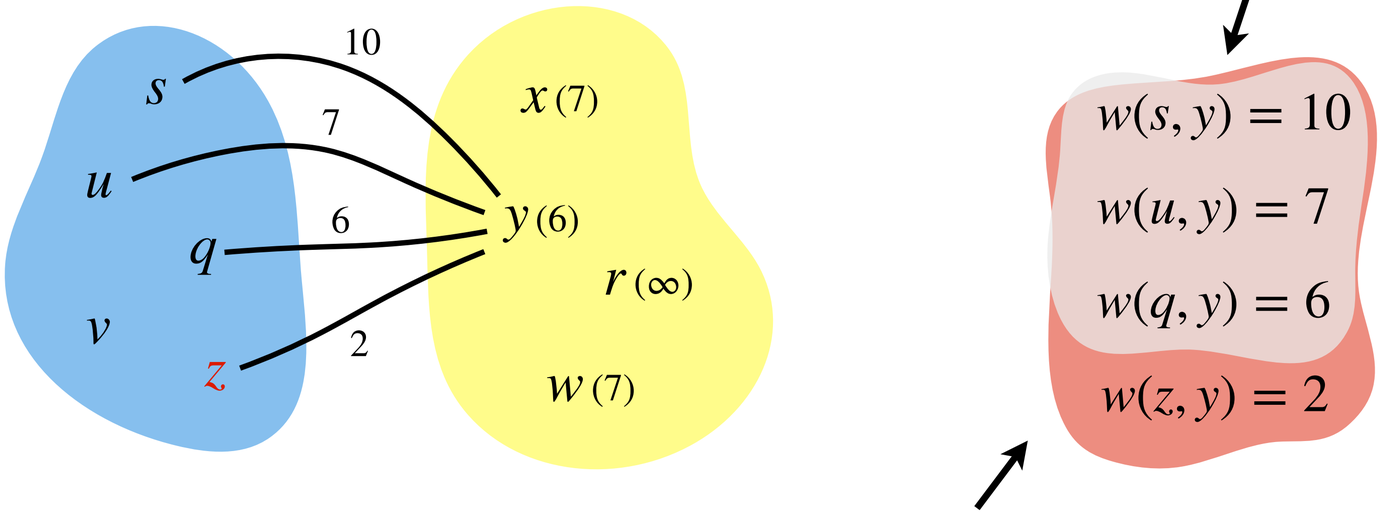
Prim's Algorithm: Implementation

Updating *keys*:



Prim's Algorithm: Implementation

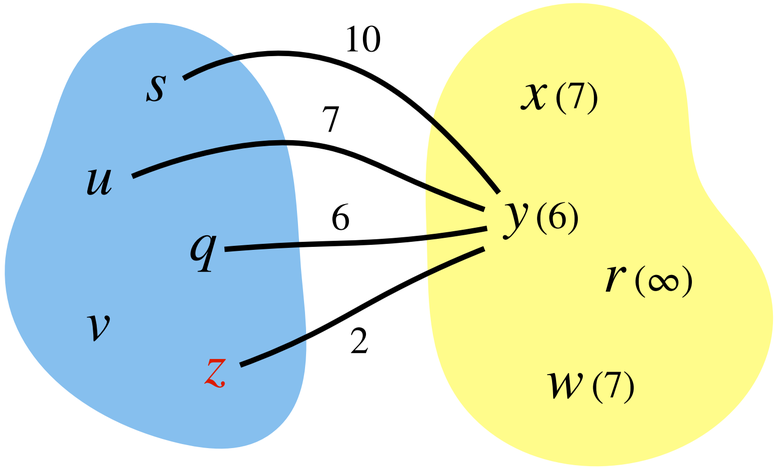
Updating *keys*:



Observation: If u is just added to tree,

Prim's Algorithm: Implementation

Updating *keys*:



Minimum of these is old $key[y]$

$w(s, y) = 10$

$w(u, y) = 7$

$w(q, y) = 6$

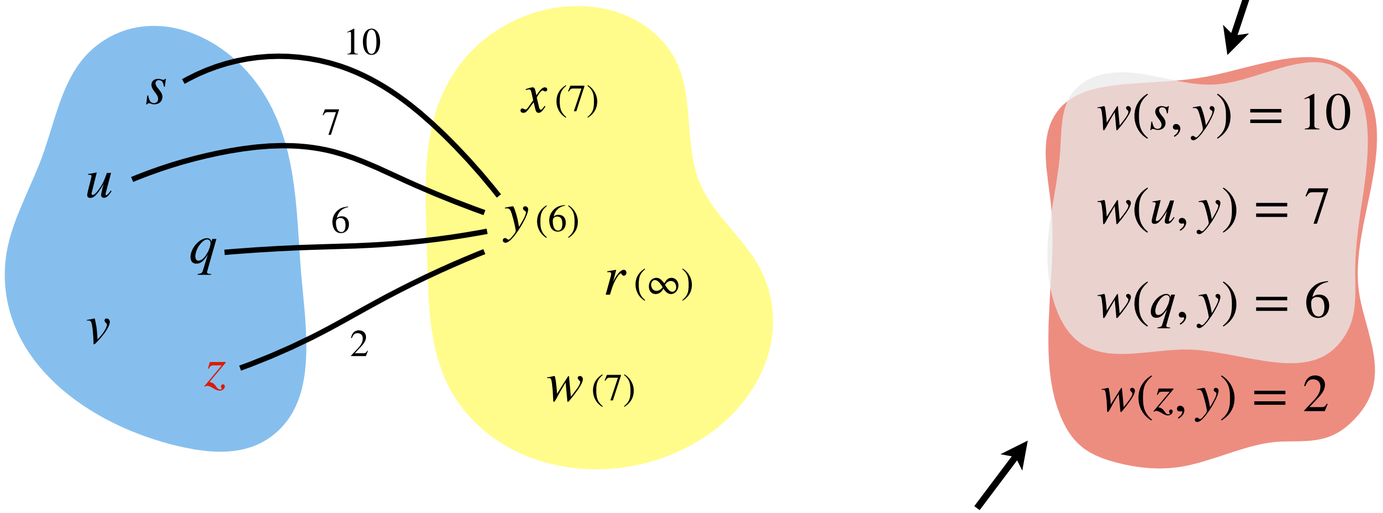
$w(z, y) = 2$

$key[y]$ will be minimum of these values.

Observation: If u is just added to tree, then $\forall v$ not in tree, such that $\{u, v\}$ is an edge

Prim's Algorithm: Implementation

Updating *keys*:

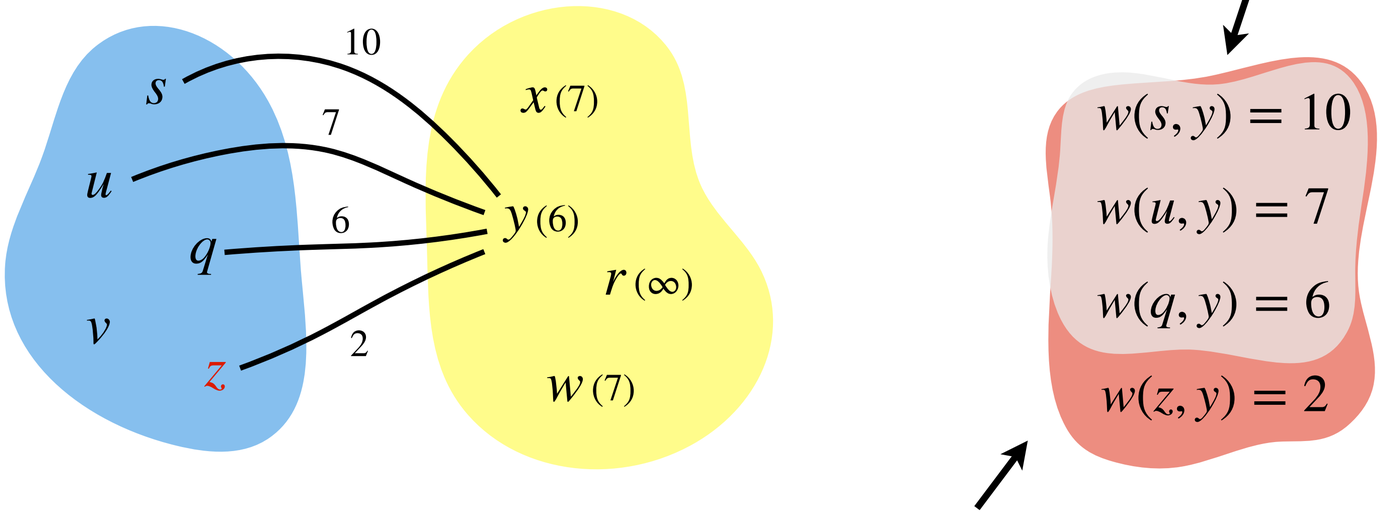


Observation: If u is just added to tree, then $\forall v$ not in tree, such that $\{u, v\}$ is an edge

$$key[v] = \mathbf{Min}(key[v], w(u, v))$$

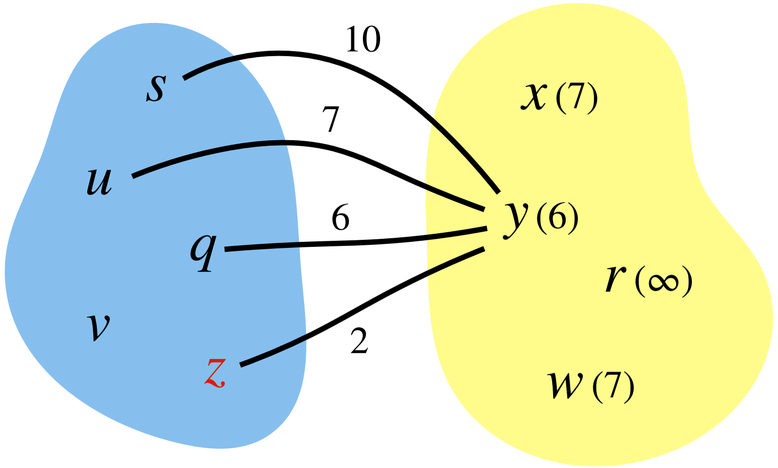
Prim's Algorithm: Implementation

Updating *keys*:



Prim's Algorithm: Implementation

Updating *keys*:



Minimum of these is old $key[y]$

$w(s, y) = 10$

$w(u, y) = 7$

$w(q, y) = 6$

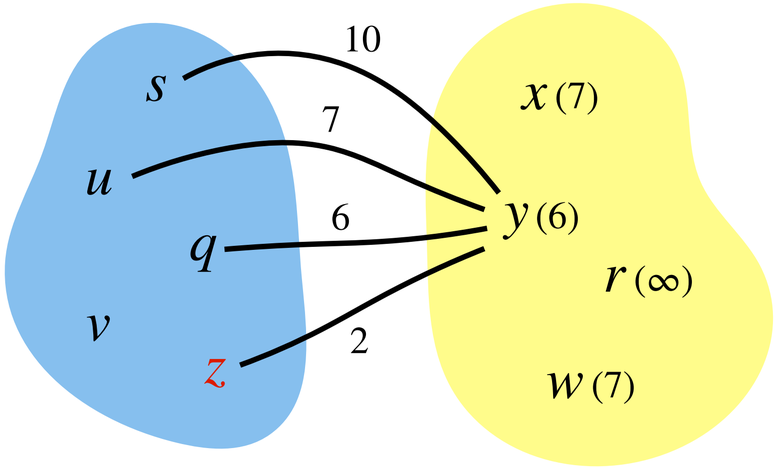
$w(z, y) = 2$

$key[y]$ will be minimum of these values.

What data structure is suitable to keep updating *keys* and removing the one with minimum?

Prim's Algorithm: Implementation

Updating *keys*:



Minimum of these is old $key[y]$

$w(s, y) = 10$

$w(u, y) = 7$

$w(q, y) = 6$

$w(z, y) = 2$

$key[y]$ will be minimum of these values.

What data structure is suitable to keep updating *keys* and removing the one with minimum?

Min-priority queue/Min-heap.

Prim’s Algorithm

Prim's Algorithm

Prim's(G, s)**:** *// create spanning tree rooted at s*

Prim's Algorithm

Prim's(G, s)**:** *// create spanning tree rooted at s*

1. Create array $key[1 : |V|]$, $parent[1 : |V|]$ *// for storing keys and parent of every vertex*

Prim's Algorithm

Prim's(G, s)**:** *// create spanning tree rooted at s*

1. Create array $key[1 : |V|]$, $parent[1 : |V|]$ *// for storing keys and parent of every vertex*
2. **for** each vertex $v \in V(G)$

Prim's Algorithm

Prim's(G, s)**:** *// create spanning tree rooted at s*

1. Create array $key[1 : |V|], parent[1 : |V|]$ *// for storing keys and parent of every vertex*
2. **for** each vertex $v \in V(G)$
3. $key[u] = \infty, parent[v] = NIL$

Prim's Algorithm

Prim's(G, s)**:** *// create spanning tree rooted at s*

1. Create array $key[1 : |V|], parent[1 : |V|]$ *// for storing keys and parent of every vertex*
2. **for** each vertex $v \in V(G)$
3. $key[u] = \infty, parent[v] = NIL$
4. $key[s] = 0, Q = \emptyset$ *// Q is the min-heap*

Prim's Algorithm

Prim's(G, s)**:** *// create spanning tree rooted at s*

1. Create array $key[1 : |V|], parent[1 : |V|]$ *// for storing keys and parent of every vertex*
2. **for** each vertex $v \in V(G)$
3. $key[u] = \infty, parent[v] = NIL$
4. $key[s] = 0, Q = \emptyset$ *// Q is the min-heap*
5. **for** each vertex $v \in V(G)$

Prim's Algorithm

Prim's(G, s)**:** *// create spanning tree rooted at s*

1. Create array $key[1 : |V|], parent[1 : |V|]$ *// for storing keys and parent of every vertex*
2. **for** each vertex $v \in V(G)$
3. $key[u] = \infty, parent[v] = NIL$
4. $key[s] = 0, Q = \emptyset$ *// Q is the min-heap*
5. **for** each vertex $v \in V(G)$
6. **Insert**$(Q, v, key[v])$ *// building the min-heap*

Prim's Algorithm

Prim's(G, s)**:** *// create spanning tree rooted at s*

1. Create array $key[1 : |V|], parent[1 : |V|]$ *// for storing keys and parent of every vertex*
2. **for** each vertex $v \in V(G)$
3. $key[u] = \infty, parent[v] = NIL$
4. $key[s] = 0, Q = \emptyset$ *// Q is the min-heap*
5. **for** each vertex $v \in V(G)$
6. **Insert**$(Q, v, key[v])$ *// building the min-heap*
7. **while** $Q \neq \emptyset$

Prim's Algorithm

Prim's(G, s)**:** *// create spanning tree rooted at s*

1. Create array $key[1 : |V|], parent[1 : |V|]$ *// for storing keys and parent of every vertex*
2. **for** each vertex $v \in V(G)$
3. $key[u] = \infty, parent[v] = NIL$
4. $key[s] = 0, Q = \emptyset$ *// Q is the min-heap*
5. **for** each vertex $v \in V(G)$
6. **Insert**$(Q, v, key[v])$ *// building the min-heap*
7. **while** $Q \neq \emptyset$
8. $u =$ **Extract-Min**(Q) *// add u to the growing tree*

Prim's Algorithm

Prim's(G, s)**:** *// create spanning tree rooted at s*

1. Create array $key[1 : |V|], parent[1 : |V|]$ *// for storing keys and parent of every vertex*
2. **for** each vertex $v \in V(G)$
3. $key[u] = \infty, parent[v] = NIL$
4. $key[s] = 0, Q = \emptyset$ *// Q is the min-heap*
5. **for** each vertex $v \in V(G)$
6. **Insert**$(Q, v, key[v])$ *// building the min-heap*
7. **while** $Q \neq \emptyset$
8. $u =$ **Extract-Min**(Q) *// add u to the growing tree*
9. **for** each vertex v in $Adj[u]$ *// update keys of u neighbours in Q*

Prim's Algorithm

Prim's(G, s)**:** *// create spanning tree rooted at s*

1. Create array $key[1 : |V|], parent[1 : |V|]$ *// for storing keys and parent of every vertex*
2. **for** each vertex $v \in V(G)$
3. $key[u] = \infty, parent[v] = NIL$
4. $key[s] = 0, Q = \emptyset$ *// Q is the min-heap*
5. **for** each vertex $v \in V(G)$
6. **Insert**$(Q, v, key[v])$ *// building the min-heap*
7. **while** $Q \neq \emptyset$
8. $u =$ **Extract-Min**(Q) *// add u to the growing tree*
9. **for** each vertex v in $Adj[u]$ *// update keys of u neighbours in Q*
10. **if** $v \in Q$ **and** $w(u, v) < key[v]$

Prim's Algorithm

Prim's(G, s)**:** *// create spanning tree rooted at s*

1. Create array $key[1 : |V|], parent[1 : |V|]$ *// for storing keys and parent of every vertex*
2. **for** each vertex $v \in V(G)$
3. $key[u] = \infty, parent[v] = NIL$
4. $key[s] = 0, Q = \emptyset$ *// Q is the min-heap*
5. **for** each vertex $v \in V(G)$
6. **Insert**$(Q, v, key[v])$ *// building the min-heap*
7. **while** $Q \neq \emptyset$
8. $u =$ **Extract-Min**(Q) *// add u to the growing tree*
9. **for** each vertex v in $Adj[u]$ *// update keys of u neighbours in Q*
10. **if** $v \in Q$ **and** $w(u, v) < key[v]$
11. $parent[v] = u, key[v] = w(u, v)$

Prim's Algorithm

Prim's(G, s)**:** *// create spanning tree rooted at s*

1. Create array $key[1 : |V|], parent[1 : |V|]$ *// for storing keys and parent of every vertex*
2. **for** each vertex $v \in V(G)$
3. $key[u] = \infty, parent[v] = NIL$
4. $key[s] = 0, Q = \emptyset$ *// Q is the min-heap*
5. **for** each vertex $v \in V(G)$
6. **Insert**$(Q, v, key[v])$ *// building the min-heap*
7. **while** $Q \neq \emptyset$
8. $u =$ **Extract-Min**(Q) *// add u to the growing tree*
9. **for** each vertex v in $Adj[u]$ *// update keys of u neighbours in Q*
10. **if** $v \in Q$ **and** $w(u, v) < key[v]$
11. $parent[v] = u, key[v] = w(u, v)$
13. **Decrease-Key**$(Q, v, w(u, v))$

Prim's Algorithm: Runtime Analysis

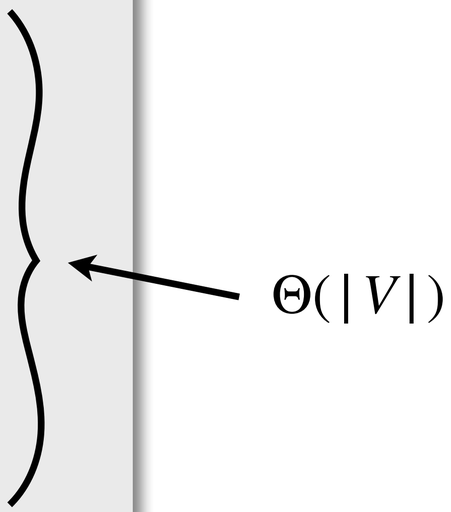
Prim's(G, r)**:**

1. Create array $key[1 : |V|], parent[1 : |V|]$
2. **for** each vertex $v \in V(G)$
3. $\quad key[u] = \infty, parent[v] = NIL$
4. $key[r] = 0, Q = \emptyset$
5. **for** each vertex $v \in V(G)$
6. $\quad$ **Insert**$(Q, v, key[v])$
7. **while** $Q \neq \emptyset$
8. $\quad u =$ **Extract-Min**(Q)
9. $\quad$ **for** each vertex v in $Adj[u]$
10. $\quad\quad$ **if** $v \in Q$ **and** $w(u, v) < key[v]$
11. $\quad\quad\quad parent[v] = u, key[v] = w(u, v)$
13. $\quad\quad\quad$ **Decrease-Key**$(Q, v, w(u, v))$

Prim's Algorithm: Runtime Analysis

Prim's(G, r)**:**

1. Create array $key[1 : |V|], parent[1 : |V|]$
2. **for** each vertex $v \in V(G)$
3. $\quad key[u] = \infty, parent[v] = NIL$
4. $key[r] = 0, Q = \emptyset$
5. **for** each vertex $v \in V(G)$
6. $\quad$ **Insert**$(Q, v, key[v])$
7. **while** $Q \neq \emptyset$
8. $\quad u =$ **Extract-Min**(Q)
9. $\quad$ **for** each vertex v in $Adj[u]$
10. $\quad\quad$ **if** $v \in Q$ **and** $w(u, v) < key[v]$
11. $\quad\quad\quad parent[v] = u, key[v] = w(u, v)$
13. $\quad\quad\quad$ **Decrease-Key**$(Q, v, w(u, v))$



Prim's Algorithm: Runtime Analysis

Prim's(G, r)**:**

1. Create array $key[1 : |V|], parent[1 : |V|]$
2. **for** each vertex $v \in V(G)$
3. $\quad key[u] = \infty, parent[v] = NIL$
4. $key[r] = 0, Q = \emptyset$
5. **for** each vertex $v \in V(G)$
6. $\quad$ **Insert**$(Q, v, key[v])$
7. **while** $Q \neq \emptyset$
8. $\quad u =$ **Extract-Min**(Q)
9. $\quad$ **for** each vertex v in $Adj[u]$
10. $\quad\quad$ **if** $v \in Q$ **and** $w(u, v) < key[v]$
11. $\quad\quad\quad parent[v] = u, key[v] = w(u, v)$
13. $\quad\quad\quad$ **Decrease-Key**$(Q, v, w(u, v))$

$\Theta(|V|)$

$O(|V| \log |V| + |E| \log |V|)$

Prim's Algorithm: Runtime Analysis

Prim's(G, r)**:**

1. Create array $key[1 : |V|], parent[1 : |V|]$
2. **for** each vertex $v \in V(G)$
3. $\quad key[u] = \infty, parent[v] = NIL$
4. $key[r] = 0, Q = \emptyset$
5. **for** each vertex $v \in V(G)$
6. $\quad$ **Insert**$(Q, v, key[v])$
7. **while** $Q \neq \emptyset$
8. $\quad u =$ **Extract-Min**(Q)
9. $\quad$ **for** each vertex v in $Adj[u]$
10. $\quad\quad$ **if** $v \in Q$ **and** $w(u, v) < key[v]$
11. $\quad\quad\quad parent[v] = u, key[v] = w(u, v)$
13. $\quad\quad\quad$ **Decrease-Key**$(Q, v, w(u, v))$

$\Theta(|V|)$

$O(|V| \log |V| + |E| \log |V|)$

Time Complexity: $O(|E| \log |V|)$